Coding Interview Guide — Volume 5

The HelloInterview gap set · Java 8+ · fully explained, with aligned diagrams

Target level: Mid-level (Easy → Hard) software-engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

What's in this volume

This volume closes the gaps between your existing Vol 1–4 and the full **HelloInterview coding catalog** (16 categories). Every problem here is one that was **not already covered** in Volumes 1–4 — no repeats. It also folds in a whole category the earlier volumes skipped: **Matrices** (spiral, rotate, set-zeroes).

For every problem you get the same detailed layout used in the enhanced Vol 1–4:

1. **Problem** — a precise, self-contained statement, with **constraints**, **clarifying assumptions** to confirm with an interviewer, and a **"Why it's tricky"** line naming the trap the problem is testing.
2. **Example** — concrete input and output.
3. **Intuition (diagram)** — the picture of the idea, in monospace-aligned ASCII.
4. **Approach (step by step)** — for every non-trivial problem: the key idea / invariant, a numbered walk-through of the algorithm, and a short worked trace on the example.
5. **Brute force (Java)** — the obvious baseline.
6. **Optimized (Java)** — the interviewer's target solution, with the trick called out.
7. **Complexity table** — time and space for both versions.

All code is **Java 8+**, compiles as-is, and favors clarity over cleverness.

The categories in this volume

#	Category	Problems
1	Matrices	Spiral Matrix, Rotate Image, Set Matrix Zeroes
2	Two Pointers & Sliding Window (new)	Sort Colors, Move Zeroes, Longest Repeating Character Replacement, Max Points From Cards
3	Stack & Linked List (new)	Decode String, Longest Valid Parentheses, Palindrome Linked List, Reorder List, Swap Nodes in Pairs
4	Trees (new)	Diameter of Binary Tree, Path Sum II, Right Side View, Zigzag Level Order, Maximum Width of Binary Tree
5	Grid & Graph DFS (new)	Surrounded Regions, Pacific Atlantic Water Flow, Graph Valid Tree
6	BFS & Backtracking (new)	Minimum Knight Moves, Bus Routes, Generate Parentheses, Word Search
7	Heap, Intervals & DP (new)	Find K Closest Elements, Employee Free Time, Maximum Profit in Job Scheduling

How to practice

- **Recognize before you code.** Which pattern is this? (Vol 1's twelve + Vol 4's five.)
- **Draw the diagram yourself** before writing Java — every problem here shows one.
- **State the brute force out loud**, then the optimization and *why* it's correct.
- **State complexity before you're asked.**

---

1. Matrices

The core idea. A 2D array is just a grid of index pairs `(row, col)`, and most matrix problems are really about **which cells you visit and in what order** — spiraling inward, swapping across a diagonal, or propagating a marker along a row/column. The recurring trick is using the matrix's own edges (four shrinking boundaries, or the first row/column) as extra bookkeeping instead of allocating a second grid.

Reach for it when: you see "traverse in some order" (spiral, diagonal, boustrophedon), "rotate/transpose in place", or "propagate a condition across a whole row and column" — and especially when the interviewer asks for O(1) extra space.

1.1 Spiral Matrix (LC 54)

Problem. Given an `m x n` matrix, return all elements of the matrix in spiral order: starting at the top-left cell, go right across the top row, down the right column, left across the bottom row, up the left column, then repeat on the shrinking inner rectangle.

- **Constraints:** `1 <= m, n <= 10`; `-100 <= matrix[i][j] <= 100`; `m` and `n` need not be equal (rectangular matrices are allowed, not just square).
- **Clarifying assumptions:** the matrix is non-empty and rectangular (every row has the same length); a single row or single column is a valid (degenerate) input.
- **Why it's tricky:** it's easy to double-count or skip cells on the last partial row/column once the spiral has shrunk to a single row or column — you must check the boundaries *between* each of the four directional sweeps, not just once per full loop.

Example.

```
Input:  [[1,2,3],
         [4,5,6],
         [7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]
```

Intuition (diagram). Track four boundaries — `top`, `bottom`, `left`, `right` — that start at the matrix's edges. Sweep along the current top row, then the current right column, then the current bottom row (if it still exists), then the current left column (if it still exists), moving the corresponding boundary inward after each sweep. Stop once the boundaries cross.

```
col:      0   1   2
       +---+---+---+
row 0  | 1 | 2 | 3 |   <- top=0
       +---+---+---+
row 1  | 4 | 5 | 6 |
       +---+---+---+
row 2  | 7 | 8 | 9 |   <- bottom=2
       +---+---+---+
         ^           ^
       left=0      right=2
```

Approach (step by step)

Key idea / invariant: maintain four boundaries `top, bottom, left, right` describing the *unvisited* rectangle. Each of the four sweeps below visits exactly the cells on one edge of that rectangle, then shrinks the boundary on that side by one. After each shrink, re-check whether the rectangle still has rows/columns left before doing the next sweep — otherwise a single remaining row or column gets walked twice.

1. While `top <= bottom` and `left <= right`: a. Walk the top row left to right: for `col` from `left` to `right`, append `matrix[top][col]`. Then `top++`. b. If `top > bottom`, the rectangle is empty — stop. Otherwise walk the right column top to bottom: for `row` from `top` to `bottom`, append `matrix[row][right]`. Then `right--`. c. If `left > right`, stop. Otherwise walk the bottom row right to left: for `col` from `right` down to `left`, append `matrix[bottom][col]`. Then `bottom--`. d. If `top > bottom`, stop. Otherwise walk the left column bottom to top: for `row` from `bottom` down to `top`, append `matrix[row][left]`. Then `left++`.
2. Repeat until the boundaries cross.

Worked trace on the 3x3 example (`top=0,bottom=2,left=0,right=2` initially):

step	sweep	cells visited	boundary update
1a	top row, `col` 0..2	1, 2, 3	`top = 1`
1b	right col, `row` 1..2	6, 9	`right = 1`
1c	bottom row, `col` 1..0	8, 7	`bottom = 1`
1d	left col, `row` 1..1	4	`left = 1`
2a	top row, `col` 1..1	5	`top = 2`
2b	check `top(2) > bottom(1)` -> stop	(none)	—

Result: `[1,2,3,6,9,8,7,4,5]` — matches the expected output.

Brute force

There isn't a meaningfully different "brute force" for spiral order — simulating direction changes with a `dr, dc` step-and-turn walk is the naive version of the same idea, without the explicit boundary bookkeeping.

```
public List<Integer> spiralOrderBrute(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length;
    boolean[][] visited = new boolean[rows][cols];
    int[] dr = {0, 1, 0, -1};   // right, down, left, up
    int[] dc = {1, 0, -1, 0};
    List<Integer> result = new ArrayList<>();
    int row = 0, col = 0, dir = 0;
    for (int i = 0; i < rows * cols; i++) {
        result.add(matrix[row][col]);
        visited[row][col] = true;
        int nr = row + dr[dir], nc = col + dc[dir];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || visited[nr][nc]) {
            dir = (dir + 1) % 4;             // turn when about to leave the grid or revisi
            nr = row + dr[dir];
            nc = col + dc[dir];
        }
        row = nr;
        col = nc;
    }
    return result;
}
```

Optimized (shrinking boundaries)

Four boundaries shrink inward after each of the four directional sweeps.

```
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> result = new ArrayList<>();
    int top = 0, bottom = matrix.length - 1;
    int left = 0, right = matrix[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int col = left; col <= right; col++) {
            result.add(matrix[top][col]);        // sweep top row left -> right
        }
        top++;
        if (top > bottom) break;

        for (int row = top; row <= bottom; row++) {
            result.add(matrix[row][right]);       // sweep right column top -> bottom
        }
        right--;
        if (left > right) break;

        for (int col = right; col >= left; col--) {
            result.add(matrix[bottom][col]);       // sweep bottom row right -> left
        }
        bottom--;
        if (top > bottom) break;

        for (int row = bottom; row >= top; row--) {
            result.add(matrix[row][left]);         // sweep left column bottom -> top
        }
        left++;
    }
    return result;
}
```

Version	Time	Space
Direction-turning simulation	O(m·n)	O(m·n) for `visited`
Shrinking boundaries	O(m·n)	O(1) extra (excluding output)

1.2 Rotate Image (LC 48)

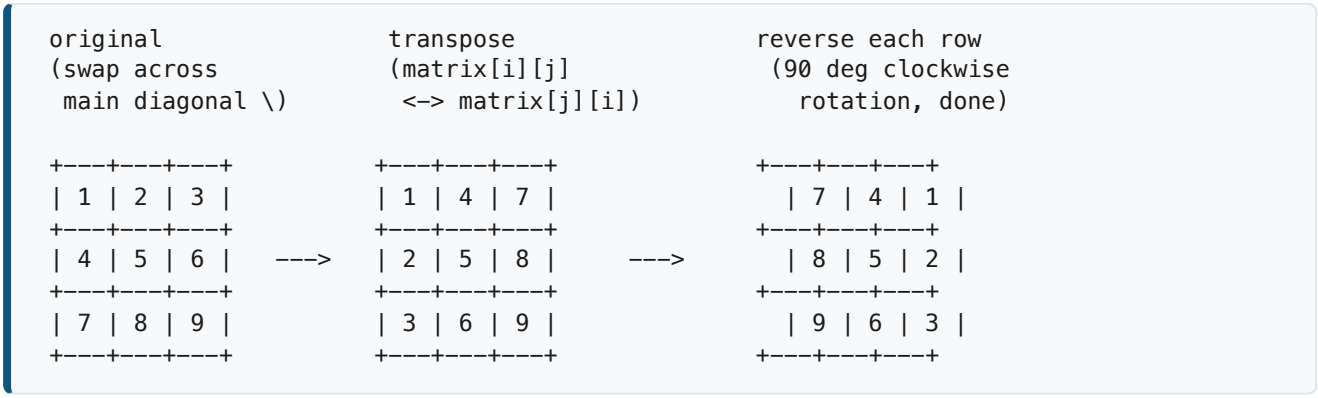
Problem. Given an `n x n` 2D matrix representing an image, rotate the image by **90 degrees clockwise**, in place — you may not allocate another 2D matrix for the rotation.

- **Constraints:** `1 <= n <= 20`; `-1000 <= matrix[i][j] <= 1000`; the matrix is always square (`n x n`).
- **Clarifying assumptions:** "in place" means O(1) extra space beyond a few scalar temporaries — allocating a second `n x n` array to build the answer and copy it back defeats the point, even though it's a valid brute force to mention first.
- **Why it's tricky:** it's tempting to rotate layer-by-layer with four-way swaps (which also works, but is fiddly to get right), when a much cleaner decomposition exists: a 90-degree clockwise rotation is exactly **transpose, then reverse each row** — recognizing that algebraic identity is the key insight, not the implementation.

Example.

```
Input:  [[1,2,3],
         [4,5,6],
         [7,8,9]]
Output: [[7,4,1],
         [8,5,2],
         [9,6,3]]
```

Intuition (diagram). Rotating 90 degrees clockwise sends `matrix[row][col]` to `result[col][n-1-row]`. Splitting that target index into two simpler moves: transpose swaps `(row, col) -> (col, row)` (flips across the main diagonal), and then reversing each row sends `(row, col) -> (row, n-1-col)`. Composing the two: `(row, col) -> (col, row) -> (col, n-1-row)`, which is exactly the 90-degree-clockwise target index.



Approach (step by step)

Key idea / invariant: a 90-degree clockwise rotation maps cell `(r, c)` to `(c, n−1−r)`. That composite map factors into two simpler, easy-to-code-in-place transformations: transpose (`(r,c) -> (c,r)`, swap across the main diagonal) followed by a horizontal flip of every row (`(r,c) -> (r, n−1−c)`). Applying the flip after the transpose gives `(r,c) -> (c,r) -> (c, n−1−r)`, matching the target exactly.

1. **Transpose in place:** for each `row` from `0` to `n−1`, and each `col` from `row+1` to `n−1` (only the upper triangle, to avoid swapping a pair twice), swap `matrix[row][col]` with `matrix[col][row]`.
2. **Reverse each row in place:** for each `row`, reverse its elements left-to-right (swap `matrix[row][left]` with `matrix[row][right]`, moving `left` inward and `right` inward until they meet).

Worked trace on the 3x3 example: - Start: `[[1,2,3],[4,5,6],[7,8,9]]`. - Transpose swaps: `(0,1)<->(1,0)` gives `4,2` swapped; `(0,2)<->(2,0)` gives `7,3` swapped; `(1,2)<->(2,1)` gives `8,6` swapped. Result after transpose: `[[1,4,7],[2,5,8],[3,6,9]]`. - Reverse row 0: `[1,4,7] -> [7,4,1]`. - Reverse row 1: `[2,5,8] -> [8,5,2]`. - Reverse row 2: `[3,6,9] -> [9,6,3]`. - Final: `[[7,4,1],[8,5,2],[9,6,3]]` — matches the expected output.

Brute force

Build a new rotated matrix using the index formula, then copy it back.

```
public void rotateBrute(int[][] matrix) {
    int n = matrix.length;
    int[][] rotated = new int[n][n];
    for (int row = 0; row < n; row++) {
        for (int col = 0; col < n; col++) {
            rotated[col][n - 1 - row] = matrix[row][col];   // direct 90-degree-clockwise m
        }
    }
    for (int row = 0; row < n; row++) {
        System.arraycopy(rotated[row], 0, matrix[row], 0, n);
    }
}
```

Optimized (transpose, then reverse each row)

Two in-place passes, no extra matrix.

```java
public void rotate(int[][] matrix) {
    int n = matrix.length;

    // Step 1: transpose across the main diagonal.
    for (int row = 0; row < n; row++) {
        for (int col = row + 1; col < n; col++) {
            int temp = matrix[row][col];
            matrix[row][col] = matrix[col][row];
            matrix[col][row] = temp;
        }
    }

    // Step 2: reverse every row left-to-right.
    for (int row = 0; row < n; row++) {
        int left = 0, right = n - 1;
        while (left < right) {
            int temp = matrix[row][left];
            matrix[row][left] = matrix[row][right];
            matrix[row][right] = temp;
            left++;
            right--;
        }
    }
}
```

Version	Time	Space
New matrix + copy back	O(n^2)	O(n^2)
Transpose + reverse rows	O(n^2)	O(1) extra

1.3 Set Matrix Zeroes (LC 73)

Problem. Given an `m x n` matrix, if an element is `0`, set its entire row and its entire column to `0`. Do it **in place**, and try to do it using O(1) extra space (beyond a couple of boolean flags).

- **Constraints:** `1 <= m, n <= 20`; `-2^31 <= matrix[i][j] <= 2^31 - 1`.
- **Clarifying assumptions:** "zero out row and column" is based on the *original* matrix's zero positions, not zeros that get introduced partway through the algorithm — so you can't just scan and mutate in a single naive pass without tracking which zeros were original.
- **Why it's tricky:** the natural O(m+n) extra-space solution (two boolean arrays marking which rows/columns contain a zero) is easy; getting to O(1) extra space means reusing the matrix's **own first row and first column** as those marker arrays, which then requires two extra flags to remember whether the first row/column themselves originally contained a zero (since you're about to overwrite them with marker data).

Example.

```
Input:  [[1,1,1],
         [1,0,1],
         [1,1,1]]
Output: [[1,0,1],
         [0,0,0],
         [1,0,1]]
```

Intuition (diagram). Instead of a separate marker array, use row 0 and column 0 of the matrix itself as the "does this row/column contain a zero" flags. Before doing that, save whether row 0 and column 0 originally had a zero in two standalone booleans — because once you start writing markers into them, you can no longer trust their original contents.

```
 original                 after marking (row/col 0          after applying markers to
                           record "row i / col j              the interior, then fixing
                           has a zero" using cell 0)          up row 0 / col 0 with flags
+---+---+---+             +---+---+---+                      +---+---+---+
| 1 | 1 | 1 |             | 1 | 0 | 1 |   <- col1 has a 0     | 1 | 0 | 1 |
+---+---+---+             +---+---+---+                      +---+---+---+
| 1 | 0 | 1 |    --->     | 0 | 0 | 1 |   <- row1 has a 0     | 0 | 0 | 0 |
+---+---+---+             +---+---+---+                      +---+---+---+
| 1 | 1 | 1 |             | 1 | 1 | 1 |                       | 1 | 0 | 1 |
+---+---+---+             +---+---+---+                      +---+---+---+
                             ^
                          row0Flag = false, col0Flag = false (neither originally had a 0)
```

Approach (step by step)

Key idea / invariant: `matrix[i][0]` will hold "row `i` contains a zero" and `matrix[0][j]` will hold "column `j` contains a zero" — but only for `i >= 1` and `j >= 1`, since `matrix[0][0]` is shared by both roles. Two separate booleans, `row0HasZero` and `col0HasZero`, capture the same information for row 0 and column 0 themselves, computed **before** those cells get reused as markers.

1. Set `row0HasZero = true` if any cell in row `0` is `0`; set `col0HasZero = true` if any cell in column `0` is `0`.
2. For `row` from `1` to `m-1`, for `col` from `1` to `n-1`: if `matrix[row][col] == 0`, mark both `matrix[row][0] = 0` and `matrix[0][col] = 0`.
3. For `row` from `1` to `m-1`, for `col` from `1` to `n-1`: if `matrix[row][0] == 0` **or** `matrix[0][col] == 0`, set `matrix[row][col] = 0`. (This reads the markers written in step 2, so it must run as a separate pass after marking is complete.)
4. If `row0HasZero`, zero out all of row `0`. If `col0HasZero`, zero out all of column `0`.

Worked trace on the 3x3 example (`matrix[1][1] = 0` is the only original zero): - Step 1: row 0 is `[1,1,1]` -> `row0HasZero = false`; column 0 is `[1,1,1]` -> `col0HasZero = false`. - Step 2: the only interior zero is at `(1,1)`, so mark `matrix[1][0] = 0` and `matrix[0][1] = 0`. Matrix is now `[[1,0,1],[0,0,1],[1,1,1]]`. - Step 3: for each interior cell, zero it if its row-marker or column-marker is `0`: - `(1,1)`: `matrix[1][0]=0` -> zero (already 0). - `(1,2)`: `matrix[1][0]=0` -> zero. Now row 1 is `[0,0,0]`. - `(2,1)`: `matrix[0][1]=0` -> zero. Now `(2,1)=0`. - `(2,2)`: `matrix[2][0]=1` and `matrix[0][2]=1` -> stays `1`. Matrix is now `[[1,0,1],[0,0,0],[1,0,1]]`. - Step 4: `row0HasZero` and `col0HasZero` are both `false`, so row 0 and column 0 are left as the marker pass computed them. - Final: `[[1,0,1],[0,0,0],[1,0,1]]` — matches the expected output.

Brute force

Record the original zero positions, then zero out their rows and columns.

```
public void setZeroesBrute(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    List<int[]> zeroPositions = new ArrayList<>();
    for (int row = 0; row < m; row++) {
        for (int col = 0; col < n; col++) {
            if (matrix[row][col] == 0) zeroPositions.add(new int[]{row, col});
        }
    }
    for (int[] pos : zeroPositions) {
        for (int col = 0; col < n; col++) matrix[pos[0]][col] = 0;   // zero the whole row
        for (int row = 0; row < m; row++) matrix[row][pos[1]] = 0;   // zero the whole colu
    }
}
```

Optimized (first row/column as markers, O(1) extra space)

Two flags remember row 0 / column 0's original state before they're overwritten as markers.

```
public void setZeroes(int[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    boolean row0HasZero = false, col0HasZero = false;

    for (int col = 0; col < n; col++) {
        if (matrix[0][col] == 0) row0HasZero = true;    // save row 0's original state
    }
    for (int row = 0; row < m; row++) {
        if (matrix[row][0] == 0) col0HasZero = true;    // save column 0's original state
    }

    for (int row = 1; row < m; row++) {
        for (int col = 1; col < n; col++) {
            if (matrix[row][col] == 0) {
                matrix[row][0] = 0;                      // mark: row `row` has a zero
                matrix[0][col] = 0;                      // mark: column `col` has a zero
            }
        }
    }

    for (int row = 1; row < m; row++) {
        for (int col = 1; col < n; col++) {
            if (matrix[row][0] == 0 || matrix[0][col] == 0) {
                matrix[row][col] = 0;                    // apply markers to the interior
            }
        }
    }

    if (row0HasZero) {
        for (int col = 0; col < n; col++) matrix[0][col] = 0;
    }
    if (col0HasZero) {
        for (int row = 0; row < m; row++) matrix[row][0] = 0;
    }
}
```

Version	Time	Space
Record positions, zero rows/cols	O(m·n·(m+n))	O(m·n) for positions
First row/col as markers	O(m·n)	O(1) extra

2. Two Pointers & Sliding Window (new)

The core idea. Instead of re-scanning the array for every candidate window or pair, keep two (or three) indices moving through the array in one coordinated pass, using an explicit invariant about what each region already means to decide when a pointer moves. Two pointers converge from opposite ends or chase each other from the same end; a sliding window keeps a contiguous span valid (or of fixed size) by growing on one side and shrinking on the other — turning an apparent O(n^2) brute force into O(n).

Reach for it when: you need to partition or sort an array in one pass with O(1) space, want the longest/shortest contiguous subarray or substring satisfying some count/frequency condition, need a fixed-size window's running sum as it slides, or must pick elements from both ends of an array under a fixed total count.

2.1 Sort Colors (LC 75)

Problem. Given an array `nums` with `n` objects colored red (`0`), white (`1`), or blue (`2`), sort them **in place** so that objects of the same color are adjacent, in the order red, white, blue. Do this **without** calling a library sort, ideally in **one pass** using **O(1)** extra space (the Dutch National Flag problem).

- **Constraints:** `n == nums.length`; `1 <= n <= 300`; each `nums[i]` is `0`, `1`, or `2`.
- **Clarifying assumptions:** values are only ever `0`, `1`, or `2` (never arbitrary integers); "sorted" means `0`s first, then `1`s, then `2`s; a two-pass counting sort is a valid warm-up answer, but the interviewer wants the one-pass, O(1)-space follow-up.
- **Why it's tricky:** there are **three** regions, not two, so a plain two-pointer partition (as in quicksort's Lomuto/Hoare scheme) isn't quite enough — you need a third pointer, and the subtle part is that the two swap cases behave **asymmetrically**: one lets you advance the middle pointer immediately, the other doesn't.

Example.

```
Input:  nums = [2, 0, 2, 1, 1, 0]
Output: [0, 0, 1, 1, 2, 2]
```

Intuition (diagram). Maintain three pointers — `low`, `mid`, `high` — that split the array into four regions: `[0, low)` is known to be all `0`s, `[low, mid)` is known to be all `1`s, `[mid, high]` is **unknown** (not yet classified), and `(high, n)` is known to be all `2`s. Below is a snapshot midway through processing the example, once the first two elements have settled into place:

```
idx:    0    1    2    3    4    5
arr:    0    0    1    1    x    2
                  L         MH
```

Here `L` marks `low = 2` (boundary of the settled `0`s / `1`s), and `M` / `H` both sit at index 4 — `mid` and `high` currently coincide, meaning only one element (`x`, still unknown) is left to classify before the array is fully sorted.

Approach (step by step)

Key idea / invariant: `nums[mid]` is always the next unclassified element. Depending on its value, either swap it into the correct settled region and shrink the unknown region from the appropriate side, or leave it (if it's already `1`, it's already in the right place).

1. Initialize `low = 0`, `mid = 0`, `high = n - 1`.
2. While `mid <= high`, look at `nums[mid]`:
3. **`nums[mid] == 0`:** swap `nums[low]` and `nums[mid]`, then `low++` and `mid++`. This is safe to advance `mid` because the value that lands at `mid` after the swap is the *old* `nums[low]` — and by the invariant, everything in `[low, mid)` is already known to be a `1` (or, if `low == mid`, the swap was a no-op on a `0`). Either way the element now at `mid` is already classified, so it's safe to move past it.
4. **`nums[mid] == 1`:** it's already in the correct region. Just `mid++`.
5. **`nums[mid] == 2`:** swap `nums[mid]` and `nums[high]`, then `high--`, but **do NOT advance `mid`**. The value that lands at `mid` after this swap came from `nums[high]`, which was part of the *unknown* region — it could be a `0`, `1`, or `2`, so it must be examined again on the next iteration.
6. When `mid > high`, every index has been classified and `nums` is sorted.

Worked trace on `[2, 0, 2, 1, 1, 0]`:

step	low	mid	high	nums[mid]	action	array after
1	0	0	5	2	swap(mid,high); high--	0,0,2,1,1,2
2	0	0	4	0	swap(low,mid); low++; mid++	0,0,2,1,1,2
3	1	1	4	0	swap(low,mid); low++; mid++	0,0,2,1,1,2
4	2	2	4	2	swap(mid,high); high--	0,0,1,1,2,2
5	2	2	3	1	mid++	0,0,1,1,2,2
6	2	3	3	1	mid++	0,0,1,1,2,2
end	2	4	3	—	mid > high, stop	0,0,1,1,2,2

Final array `[0, 0, 1, 1, 2, 2]` matches the expected output.

Brute force

Ignore the "only 3 values" structure and use a generic comparison sort.

```
public void sortColorsBrute(int[] nums) {
    java.util.Arrays.sort(nums);   // O(n log n) — doesn't exploit the 3-value structure
}
```

Optimized (Dutch National Flag, one pass)

Three pointers carve the array into settled-0s / settled-1s / unknown / settled-2s regions.

```
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums, low, mid);
            low++;
            mid++;                  // value swapped into mid is already classified (0 or 1)
        } else if (nums[mid] == 1) {
            mid++;                  // 1 already belongs in the middle region
        } else {                    // nums[mid] == 2
            swap(nums, mid, high);
            high--;                 // do NOT advance mid: swapped-in value from high is sti
        }
    }
}

private void swap(int[] nums, int i, int j) {
    int tmp = nums[i];
    nums[i] = nums[j];
    nums[j] = tmp;
}
```

Version	Time	Space
Library sort	O(n log n)	O(1)
Dutch National Flag	O(n)	O(1)

2.2 Move Zeroes (LC 283)

Problem. Given an integer array `nums`, move all `0`s to the end of the array **in place** while maintaining the relative order of the non-zero elements. Do this without making a copy of the array.

- **Constraints:** `1 <= nums.length <= 10^4`; `-2^31 <= nums[i] <= 2^31 - 1`.
- **Clarifying assumptions:** must be done in place with O(1) extra space; relative order of the non-zero elements must be preserved (only the zeros' positions change).
- **Why it's tricky:** it's mostly a warm-up — the only trap is trying to delete-and-shift elements one at a time, which is O(n^2); the fix is a single write-pointer scan.

Example.

```
Input:  nums = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

Intuition (diagram). Scan with `i`; keep a second pointer `w` marking the next slot that should hold a non-zero value. Whenever `nums[i]` is non-zero, swap it into slot `w` and advance `w`. Zeros get pushed to the right automatically as a side effect.

```
idx: 0    1    2    3    4
arr: 0    1    0    3    12
i,w
```

After processing, `w` has advanced to the boundary between non-zero and zero values:

```
idx: 0    1    2    3    4
arr: 1    3    12   0    0
                    w
```

Approach (brief)

1. Initialize `writePos = 0`.
2. Scan `i` from `0` to `n - 1`. Whenever `nums[i] != 0`, swap `nums[i]` and `nums[writePos]`, then `writePos++`.
3. Every non-zero value gets written, in order, starting at index `0`; everything after the final `writePos` is left as (or becomes) `0`.

Brute force

Copy non-zeros into a new array in order, then pad the rest with zeros.

```
public void moveZeroesBrute(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    int idx = 0;
    for (int num : nums) {                  // copy non-zeros first, preserving order
        if (num != 0) result[idx++] = num;
    }
    while (idx < n) result[idx++] = 0;      // pad the rest with zeros
    System.arraycopy(result, 0, nums, 0, n);
}
```

Optimized (single write-pointer scan, in place)

Swap each non-zero into the next open "front" slot as it's found.

```
public void moveZeroes(int[] nums) {
    int writePos = 0; // next index that should hold a non-zero value
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] != 0) {
            int tmp = nums[writePos];
            nums[writePos] = nums[i];
            nums[i] = tmp;
            writePos++;
        }
    }
}
```

Version	Time	Space
Copy to new array	O(n)	O(n)
Write-pointer swap	O(n)	O(1)

2.3 Longest Repeating Character Replacement (LC 424)

Problem. Given a string `s` of uppercase English letters and an integer `k`, you may replace **at most** `k` characters of `s` with any other uppercase letter. Return the length of the **longest substring** containing only one repeated letter after performing (at most) `k` such replacements.

- **Constraints:** `1 <= s.length <= 10^5`; `s` consists of only uppercase English letters; `0 <= k <= s.length`.
- **Clarifying assumptions:** a replacement can turn a character into *any* other character (not just an adjacent letter); we only need the **length**, not the substring itself; using fewer than `k` replacements is allowed.
- **Why it's tricky:** the natural sliding-window check is "can this window be made single-letter within `k` replacements?", i.e. `windowLength - mostFrequentCharCount <= k`. The subtlety is that the running "most frequent count so far" (`maxFreq`) is **never decremented** even as the window slides past characters — and that's still correct, because the window length is monotonically non-decreasing: it only grows when a genuinely higher frequency is found (which recomputes `maxFreq` accurately at that moment), and otherwise it slides forward at a fixed length rather than shrinking below a length already achieved.

Example.

```
Input:  s = "AABABBA", k = 1
Output: 4
```

Intuition (diagram). Grow the window by moving `right`. Each new character updates `maxFreq` to the best single-letter count seen in the window. When `windowLength - maxFreq` exceeds `k`, the window can no longer be fixed with `k` replacements, so `left` advances by one — the window slides forward at the same length instead of shrinking below its best size.

At `right = 4` the window `[0, 4]` = `"AABAB"` has just become invalid (`length 5 - maxFreq 3 = 2 > k`), forcing a shift:

```
idx:    0    1    2    3    4    5    6
chr:    A    A    B    A    B    B    A
before: L    -    -    -    R
after:       L    -    -    R
```

`before` is the invalid window `[0, 4]`; `after` is the same-length-minus-one window `[1, 4]` once `left` has advanced to fix the violation.

Approach (step by step)

Key idea / invariant: `maxFreq` is a non-decreasing high-water mark of "the largest count of any single letter seen in a window of the current size (or a size we've already passed)." `maxLen` is only ever updated at the instant a window is genuinely valid — and that instant always coincides with `count[s.charAt(right)]` having just been incremented, so `maxFreq` is exactly accurate for that letter at that moment. Later, as the window slides, `maxFreq` can become "stale" (larger than the true current max), but that's harmless: it can only make the algorithm skip a shrink it technically didn't need for

`maxLen`'s purposes, because growing *past* the recorded `maxLen` still requires a real, freshly incremented count to exceed it.

1. Keep a 26-entry `count` array, `left = 0`, `maxFreq = 0`, `maxLen = 0`.
2. For each `right` from `0` to `n - 1`: a. Increment `count[s.charAt(right)]`; update `maxFreq = max(maxFreq, count[s.charAt(right)])`. b. While `(right - left + 1) - maxFreq > k`: this window can't be fixed with `k` replacements, so decrement `count[s.charAt(left)]` and `left++`. c. Update `maxLen = max(maxLen, right - left + 1)`.
3. Return `maxLen`.

Worked trace on `"AABABBA"`, `k = 1`:

right	char	count updated	maxFreq	window (len)	len - maxFreq	shrink?	maxLen
0	A	A=1	1	[0,0] (1)	0	no	1
1	A	A=2	2	[0,1] (2)	0	no	2
2	B	B=1	2	[0,2] (3)	1	no	3
3	A	A=3	3	[0,3] (4)	1	no	4
4	B	B=2	3	[0,4] (5)	2	yes -> [1,4] (4)	4
5	B	B=3	3	[1,5] (5)	2	yes -> [2,5] (4)	4
6	A	A=2 (after shrinks)	3	[3,6] (4)	1	no	4

Final `maxLen = 4`, matching the expected output.

Brute force

Check every substring directly, recomputing letter counts for each starting point.

```
public int characterReplacementBrute(String s, int k) {
    int n = s.length();
    int maxLen = 0;
    for (int left = 0; left < n; left++) {
        int[] count = new int[26];
        int maxFreq = 0;
        for (int right = left; right < n; right++) {
            count[s.charAt(right) - 'A']++;
            maxFreq = Math.max(maxFreq, count[s.charAt(right) - 'A']);
            int windowLen = right - left + 1;
            if (windowLen - maxFreq <= k) {
                maxLen = Math.max(maxLen, windowLen);
            }
        }
    }
    return maxLen;
}
```

Optimized (sliding window with a high-water-mark maxFreq)

Grow the window; slide (never truly shrink below the best length) when it becomes invalid.

```
public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int left = 0, maxFreq = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        int c = s.charAt(right) - 'A';
        count[c]++;
        maxFreq = Math.max(maxFreq, count[c]);          // best single-letter count seen sc
        while ((right - left + 1) - maxFreq > k) {       // window needs more than k replac
            count[s.charAt(left) - 'A']--;
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Version	Time	Space
Check every substring	O(n^2)	O(1) (26-array)
Sliding window	O(n)	O(1) (26-array)

2.4 Max Points You Can Obtain From Cards (LC 1423)

Problem. There are several cards arranged in a row, each with a point value in `cardPoints`. In one step you may take **one** card from the beginning or the end of the row. You must take **exactly** `k` cards total. Return the **maximum** total points you can obtain.

- **Constraints:** `1 <= cardPoints.length <= 10^5`; `1 <= cardPoints[i] <= 10^4`; `1 <= k <= cardPoints.length`.
- **Clarifying assumptions:** exactly `k` cards must be taken (not "up to `k`"); cards can only be taken from the two ends, never the middle; the order in which you take them doesn't affect the total, only *which* cards end up taken.
- **Why it's tricky:** it looks like a two-pointer "greedily take the bigger end" problem, but greedy fails (a small card now can unlock much bigger cards later). The key reframing: any sequence of `k` end-picks (some prefix count `i` from the left, `k - i` from the right) leaves exactly one **contiguous window of size** `n - k` untouched in the middle. So maximizing the taken sum is equivalent to **minimizing the sum of a fixed-size** `(n - k)` **sliding window** — turning "pick from both ends" into a classic fixed-size window problem.

Example.

```
Input:  cardPoints = [1, 2, 3, 4, 5, 6, 1], k = 3
Output: 12          (take the 3 rightmost cards: 5 + 6 + 1 = 12)
```

Intuition (diagram). `n = 7`, `k = 3`, so the untouched middle window has fixed size `n − k = 4`. Minimizing that window's sum is the same problem as maximizing the taken sum. For this example the minimum-sum window turns out to be the leftmost 4 cards, which means the optimal pick is the 3 cards on the right:

```
idx: 0    1    2    3    4    5    6
val: 1    2    3    4    5    6    1
reg: U    U    U    U    T    T    T
```

(`U` = left in the untouched window, `T` = taken.)

Approach (step by step)

Key idea: total taken sum = `total(cardPoints) − sum(untouched window)`. The untouched window always has fixed size `windowSize = n − k` (however the picks were split between the two ends, exactly that many cards are left in the middle), so slide a fixed-size window of that length across the array and find its **minimum** sum.

1. Compute `total`, the sum of all of `cardPoints`.
2. Let `windowSize = n − k`. If `windowSize == 0` (i.e. `k == n`), return `total` — every card is taken.
3. Compute the sum of the first window, `cardPoints[0 .. windowSize − 1]`; that's the initial `minWindowSum`.
4. Slide the window one step at a time from `windowSize` to `n − 1`: add the new right edge, subtract the edge leaving on the left, and update `minWindowSum` if this window is smaller.
5. Return `total − minWindowSum`.

Worked trace on `[1, 2, 3, 4, 5, 6, 1]`, `k = 3` (`windowSize = 4`, `total = 22`):

window (indices)	cards	sum	minSoFar
[0,3]	1,2,3,4	10	10
[1,4]	2,3,4,5	14	10
[2,5]	3,4,5,6	18	10
[3,6]	4,5,6,1	16	10

`minWindowSum = 10`, so the answer is `total − minWindowSum = 22 − 10 = 12`, matching the expected output.

Brute force

Try every split of `i` cards from the left and `k − i` from the right, summing directly.

```
public int maxScoreBrute(int[] cardPoints, int k) {
    int n = cardPoints.length;
    int best = 0;
    for (int i = 0; i <= k; i++) {          // i cards from the left, k-i from the right
        int sum = 0;
        for (int j = 0; j < i; j++) sum += cardPoints[j];
        for (int j = 0; j < k - i; j++) sum += cardPoints[n - 1 - j];
        best = Math.max(best, sum);
    }
    return best;
}
```

Optimized (minimize the fixed-size untouched window)

Slide a window of size `n - k`; the answer is `total` minus that window's minimum sum.

```
public int maxScore(int[] cardPoints, int k) {
    int n = cardPoints.length;
    int windowSize = n - k;                 // size of the untouched middle window
    int total = 0;
    for (int point : cardPoints) total += point;

    if (windowSize == 0) return total;      // k == n: every card is taken

    int windowSum = 0;
    for (int i = 0; i < windowSize; i++) windowSum += cardPoints[i];  // first window
    int minWindowSum = windowSum;

    for (int i = windowSize; i < n; i++) {
        windowSum += cardPoints[i] - cardPoints[i - windowSize];      // slide window right
        minWindowSum = Math.min(minWindowSum, windowSum);
    }

    return total - minWindowSum;            // taken sum = total - sum of cards left behind
}
```

Version	Time	Space
Try every split (direct sums)	O(k^2)	O(1)
Minimize fixed-size window	O(n)	O(1)

---

3. Stack & Linked List (new)

The core idea. A stack turns nested, "innermost-first" structure — brackets, matching parens, recursive decoding — into an iterative push/pop game where the top of the stack always holds the most recent unresolved context. A singly linked list turns pointer-chasing into geometry: the classic **fast/slow pointer** finds the middle in one pass, and reversing a sublist in place is just re-wiring three pointers (`prev` , `curr` , `next`) as you walk.

Reach for it when: you see nested brackets or run-length encodings (stack); "longest valid substring of matching symbols" (stack with a sentinel, or a two-pass counter); or anything on a singly linked list that asks for the middle, a reversal, a reordering, or a pairwise swap — those almost always combine fast/slow pointers with in-place reversal.

Definitions used throughout this section (a standard singly linked list node):

```
class ListNode {
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}
```

3.1 Decode String (LC 394)

Problem. Given an encoded string `s` in the format `k[encoded_string]` , where the `encoded_string` inside the square brackets is repeated exactly `k` times, decode it and return the resulting string. Encodings can be **nested** (e.g. `3[a2[c]]`). `k` is guaranteed to be a positive integer.

- **Constraints:** `1 <= s.length <= 30` ; `s` consists of lowercase English letters, digits `1–9` , and the characters `[` `]` ; the decoded output length fits comfortably in memory (LeetCode caps it around `10^5`); `s` is guaranteed to be a valid encoding.
- **Clarifying assumptions:** digits can be multi-digit (e.g. `12[a]`), so numbers must be accumulated digit-by-digit, not read as a single character; there are no plain (unbracketed) digits outside of a `k[...]` pattern.
- **Why it's tricky:** the nesting means you can't just scan once — a `]` you hit might close a block whose *repeat count* and *prefix string* were set aside long before, so you need a mechanism that remembers "what was being built before this bracket opened," which is exactly what a stack is for.

Example.

```
Input:  s = "3[a2[c]]"
Output: "accaccacc"
```

Intuition (diagram). Keep two stacks: one for pending repeat **counts**, one for the **string built so far** each time a `[` is opened. When you see `[` , push the current number and the current partial string, then

reset both to start fresh inside the brackets. When you see `]`, pop a count and a prefix, and fold the just-finished inner string back into that prefix, repeated `count` times.

```
chars:  3    [    a    2    [    c    ]    ]

after '3' : num=3
after '[' : countStack=[3]      stringStack=[""]      cur=""
after 'a' : cur="a"
after '2' : num=2
after '[' : countStack=[3,2]    stringStack=["","a"]  cur=""
after 'c' : cur="c"
after ']' : pop (2,"a")  ->  cur = "a" + "c"*2       = "acc"
after ']' : pop (3,"")   ->  cur = ""  + "acc"*3     = "accaccacc"
```

Approach (step by step)

Key idea: a `[` always means "freeze my current progress and start a fresh scope"; a `]` always means "the scope that just closed needs to be repeated and glued back onto whatever was frozen before it." A stack naturally remembers scopes in the right (innermost-first) order.

1. Maintain `currentString` (the string built in the current scope), `currentNum` (the digits accumulated so far), a `countStack`, and a `stringStack`.
2. Scan `s` left to right: a. If the character is a digit, fold it into `currentNum` (`currentNum = currentNum*10 + d`) — this handles multi-digit counts. b. If the character is `[`, push `currentNum` onto `countStack` and `currentString` onto `stringStack`; then reset `currentNum = 0` and `currentString = ""` to start the new inner scope. c. If the character is `]`, pop `count` from `countStack` and `prevString` from `stringStack`; set `currentString = prevString + currentString.repeat(count)` — the inner scope's result gets repeated and appended to what came before it. d. Otherwise (a lowercase letter), append it to `currentString`.
3. After the scan, `currentString` holds the fully decoded result.

Worked trace on `"3[a2[c]]"` (see diagram above): after the first `]` the inner `"c"` is repeated twice and glued onto the frozen prefix `"a"`, giving `"acc"`. After the second `]` that `"acc"` is repeated three times and glued onto the frozen prefix `""`, giving the final `"accaccacc"`.

Brute force

Recursively find matching brackets and expand them; correct but involves repeated string scanning and substring extraction.

```
public String decodeStringBrute(String s) {
    return decodeHelper(s, new int[]{0});
}

private String decodeHelper(String s, int[] pos) {
    StringBuilder result = new StringBuilder();
    while (pos[0] < s.length() && s.charAt(pos[0]) != ']') {
        char c = s.charAt(pos[0]);
        if (Character.isDigit(c)) {
            int num = 0;
            while (Character.isDigit(s.charAt(pos[0]))) {
                num = num * 10 + (s.charAt(pos[0]) - '0');
                pos[0]++;
            }
            pos[0]++; // skip '['
            String inner = decodeHelper(s, pos);
            pos[0]++; // skip ']'
            for (int i = 0; i < num; i++) result.append(inner);
        } else {
            result.append(c);
            pos[0]++;
        }
    }
    return result.toString();
}
```

Optimized (two stacks: counts and partial strings)

Iterative, single pass, O(1) extra bookkeeping per bracket.

```
public String decodeString(String s) {
    Deque<Integer> countStack = new ArrayDeque<>();
    Deque<String> stringStack = new ArrayDeque<>();
    String currentString = "";
    int currentNum = 0;

    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            currentNum = currentNum * 10 + (c - '0');       // accumulate multi-digit count
        } else if (c == '[') {
            countStack.push(currentNum);                    // freeze the count
            stringStack.push(currentString);                // freeze the prefix so far
            currentNum = 0;
            currentString = "";
        } else if (c == ']') {
            int count = countStack.pop();
            String prevString = stringStack.pop();
            currentString = prevString + currentString.repeat(count); // repeat and re-atta
        } else {
            currentString += c;
        }
    }
    return currentString;
}
```

Version	Time	Space
Recursive descent	O(maxK · n)	O(n) recursion depth

Version	Time	Space
Two stacks	O(maxK · n)	O(n)

3.2 Longest Valid Parentheses (LC 32, Hard)

Problem. Given a string `s` containing only the characters `(` and `)`, find the length of the longest substring of `s` that is a valid (well-formed) parentheses sequence.

- **Constraints:** `0 <= s.length <= 3*10^4`; `s` consists only of `(` and `)`.
- **Clarifying assumptions:** an empty string has a longest valid substring of length `0`; the substring must be **contiguous**; a single valid pair anywhere in the middle of an otherwise invalid string still counts.
- **Why it's tricky:** it's easy to detect *whether* a full string is balanced with a simple counter, but here you need the length of the longest **contiguous** valid run, which requires knowing, at each `)`, exactly where the current valid run started — that's what a stack of indices (with a clever sentinel base) or a two-pass counter trick gives you.

Example.

```
Input:  s = ")()())"
Output: 4          (the substring "()()" starting at index 1)
```

Intuition (diagram). Push the **index** of every `(`. When a `)` arrives, pop — that pop represents matching the most recent open paren. If the stack becomes empty after popping, this `)` is unmatched, so push its own index as a new "wall" that future valid runs are measured from. Otherwise, the new top of the stack is the index **just before** the current valid run, so `i - stack.top()` is the length of that run. A `-1` sentinel is pushed at the very start so this formula works even for a valid run beginning at index `0`.

```
index :  0    1    2    3    4    5
char  :  )    (    )    (    )    )

stack starts as [-1]                              maxLen = 0
i=0 ')' : pop -1 -> empty -> push 0     stack=[0]        maxLen=0
i=1 '(' : push 1                       stack=[0,1]      maxLen=0
i=2 ')' : pop 1 -> top=0  len=2-0=2     stack=[0]        maxLen=2
i=3 '(' : push 3                       stack=[0,3]      maxLen=2
i=4 ')' : pop 3 -> top=0  len=4-0=4     stack=[0]        maxLen=4
i=5 ')' : pop 0 -> empty -> push 5      stack=[5]        maxLen=4
```

Approach (step by step)

Key idea / invariant: the stack always holds the indices of parentheses that are *not yet matched* — after the sentinel, every remaining index on the stack is either an unmatched `(` or an unmatched `)` acting as a wall. Whenever a `)` successfully matches (the stack is non-empty after popping), the new top is the boundary of the current valid run, so subtracting it from the current index gives that run's length directly, with no need to track substrings.

1. Initialize a stack with a single sentinel value `-1` (a "virtual wall" one position before the string starts) and `maxLen = 0`.
2. For each index `i` and character `c` in `s`: a. If `c == '('`, push `i`. b. Else (`c == ')'`): pop the top of the stack.
    - If the stack is now **empty**, this `)` has nothing to match against; push `i` itself as the new wall.
    - Otherwise, update `maxLen = max(maxLen, i - stack.peek())` — the run from just after the new top through `i` is valid.
3. Return `maxLen`.

Worked trace on `")()())"`: see the diagram above — the stack empties out and gets a new wall at index `0` after the first `)`, then two matches in a row (`()` at indices 1-2, then extending to `()()` at indices 1-4) push `maxLen` up to `4`, and the trailing `)` at index 5 starts a fresh, unmatched wall that never extends further. Final answer: `4`.

Brute force

For every substring, check whether it's balanced; keep the longest that is.

```
public int longestValidParenthesesBrute(String s) {
    int maxLen = 0;
    for (int i = 0; i < s.length(); i++) {
        for (int j = i + 2; j <= s.length(); j += 2) { // valid length must be even
            if (isValid(s.substring(i, j))) {
                maxLen = Math.max(maxLen, j - i);
            }
        }
    }
    return maxLen;
}

private boolean isValid(String s) {
    int balance = 0;
    for (char c : s.toCharArray()) {
        balance += (c == '(') ? 1 : -1;
        if (balance < 0) return false;
    }
    return balance == 0;
}
```

Optimized (stack of indices with a -1 sentinel)

One pass; the sentinel lets the length formula work uniformly, even for a run starting at index 0.

```
public int longestValidParentheses(String s) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(-1);                          // sentinel: virtual wall before index 0
    int maxLen = 0;

    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == '(') {
            stack.push(i);
        } else {
            stack.pop();                     // attempt to match with the last unmatched '('
            if (stack.isEmpty()) {
                stack.push(i);                // no match: this ')' becomes the new wall
            } else {
                maxLen = Math.max(maxLen, i - stack.peek());
            }
        }
    }
    return maxLen;
}
```

Version	Time	Space
Check every substring	O(n^3)	O(n)
Stack of indices	O(n)	O(n)

3.3 Palindrome Linked List (LC 234)

Problem. Given the `head` of a singly linked list, determine whether it is a palindrome (reads the same forwards and backwards). Aim for O(n) time and, ideally, O(1) extra space.

- **Constraints:** `1 <= list length <= 10^5`; node values are integers in `0..9` (typical bound, though the technique works for any comparable value).
- **Clarifying assumptions:** it's acceptable (and typical for the O(1)-space solution) to temporarily mutate the list's pointers, as long as the function still returns the correct boolean; a list of length 1 is trivially a palindrome.
- **Why it's tricky:** singly linked lists can't be walked backwards, so the obvious "compare from both ends" trick from arrays doesn't apply directly. The O(1)-space fix is to physically **reverse the second half in place** so it can be walked forward like the first half — which requires first finding the middle without knowing the length in advance.

Example.

```
Input:  1 -> 2 -> 2 -> 1
Output: true
```

Intuition (diagram). Use the classic **fast/slow pointer** trick to find the middle in one pass (`fast` moves two steps for every one `slow` takes), reverse the list from the middle onward, then walk the first half and the reversed second half together, comparing values.

```
nodes:    [1]------[2]------[2]------[1]------null
index:      1         2         3         4

slow/fast trace:
  start     slow=1   fast=1
  iter 1    slow=2   fast=3
  iter 2    slow=3   fast=null   (fast.next was null -> loop stops)

slow stops at index 3 -> second half begins there

first half  (unchanged):     [1]------[2]------null
second half (before reverse):[2]------[1]------null
second half (after reverse): [1]------[2]------null

compare pair by pair:
p1: [1]------[2]------null
p2: [1]------[2]------null
     match     match
all pairs equal -> palindrome (true)
```

Approach (step by step)

Key idea / invariant: the fast/slow pointer pair guarantees that when `fast` runs off the end, `slow` sits at the first node of the "second half" (inclusive of the middle for odd lengths). Reversing from `slow` onward turns that second half into a forward-walkable list whose values, read front to back, are the original list's values read back to front.

1. Find the middle: `slow = head`, `fast = head`; while `fast != null && fast.next != null`, set `slow = slow.next` and `fast = fast.next.next`.
2. Reverse the sublist starting at `slow` (standard iterative pointer reversal), producing `secondHalfHead`.
3. Walk `p1` from `head` and `p2` from `secondHalfHead` together, comparing `p1.val == p2.val` at each step, for as many steps as the second half has nodes.
4. If every comparison matches, the list is a palindrome; otherwise it is not. (Optionally, re-reverse and re-attach the second half to restore the original list.)

Worked trace on `1 -> 2 -> 2 -> 1`: `slow` ends at the node holding the third value (the second `2`), so the second half `2 -> 1` reverses to `1 -> 2`. Comparing `head` (`1 -> 2`) against the reversed second half (`1 -> 2`) gives two matches in a row, so the list is a palindrome. Matches the expected output `true`.

Brute force

Copy all values into an array (or `ArrayList`), then compare from both ends — easy, but not O(1) space.

```
public boolean isPalindromeBrute(ListNode head) {
    List<Integer> values = new ArrayList<>();
    for (ListNode node = head; node != null; node = node.next) {
        values.add(node.val);
    }
    int left = 0, right = values.size() - 1;
    while (left < right) {
        if (!values.get(left).equals(values.get(right))) return false;
        left++;
        right--;
    }
    return true;
}
```

Optimized (fast/slow to find middle, reverse second half in place)

O(1) extra space beyond a few pointers.

```
public boolean isPalindrome(ListNode head) {
    if (head == null || head.next == null) return true;

    // 1) find the start of the second half
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // 2) reverse the second half in place
    ListNode secondHalf = reverse(slow);

    // 3) compare first half against reversed second half
    ListNode p1 = head, p2 = secondHalf;
    boolean result = true;
    while (p2 != null) {
        if (p1.val != p2.val) {
            result = false;
            break;
        }
        p1 = p1.next;
        p2 = p2.next;
    }
    return result;
}

private ListNode reverse(ListNode node) {
    ListNode prev = null;
    while (node != null) {
        ListNode next = node.next;
        node.next = prev;   // relink backwards
        prev = node;
        node = next;
    }
    return prev;
}
```

Version	Time	Space
Copy to array	O(n)	O(n)
Fast/slow + in-place reverse	O(n)	O(1)

3.4 Reorder List (LC 143)

Problem. Given the `head` of a singly linked list `L0 -> L1 -> ... -> Ln-1 -> Ln`, reorder it in place to `L0 -> Ln -> L1 -> Ln-1 -> L2 -> Ln-2 -> ...`. You may not modify the values in the nodes — only the node links.

- **Constraints:** `1 <= list length <= 5*10^4`; node values are typical 32-bit integers.
- **Clarifying assumptions:** the reordering must be done **in place** (O(1) extra space beyond a constant number of pointers); a list of length 1 or 2 is already in the required order (or requires no visible change).
- **Why it's tricky:** the target order interleaves the list with itself **reversed**, which means you need to walk from both ends at once on a structure that only supports forward traversal — the fix is the same fast/slow-then-reverse combo as the palindrome problem, followed by a careful alternating merge.

Example.

```
Input:  1 -> 2 -> 3 -> 4 -> 5
Output: 1 -> 5 -> 2 -> 4 -> 3
```

Intuition (diagram). Split the list into two halves at the middle, reverse the second half, then zip the two halves together one node at a time — first half node, then second half node, alternating.

```
nodes:   [1]---[2]---[3]---[4]---[5]---null
index:     1     2     3     4     5

slow/fast trace:
  start   slow=1  fast=1
  iter1   slow=2  fast=3
  iter2   slow=3  fast=5
  iter3   fast.next=null -> stop

slow stops at index 3 -> split right after it:
  first half : [1]---[2]---[3]---null
  second half: [4]---[5]---null

reverse second half:
  second half (reversed): [5]---[4]---null

zip alternately (first, second, first, second, ...):
  1 -> 5 -> 2 -> 4 -> 3 -> null
```

Approach (step by step)

Key idea: this is three well-known sub-routines chained together: (1) find the middle with fast/slow, (2) reverse the back half in place, (3) merge two lists by alternating nodes instead of comparing values.

1. Find the middle: `slow = head`, `fast = head`; while `fast != null && fast.next != null`, advance `slow` by one and `fast` by two. `slow` ends at the last node of the first half (for the merge step below, cut the list right after `slow`).
2. Cut the list into `first = head` and `second = slow.next`; set `slow.next = null` to terminate the first half.
3. Reverse `second` in place (same helper as 3.3) to get `secondReversed`.
4. Merge `first` and `secondReversed` by alternating: repeatedly take the next node from `first`, then the next node from `secondReversed`, relinking as you go, until one list is exhausted.
5. The merged chain, starting at the original `head`, is the reordered list.

Worked trace on `1 -> 2 -> 3 -> 4 -> 5`: the middle-finding stops with `slow` at node `3`, so the split gives `first = 1 -> 2 -> 3` and `second = 4 -> 5`. Reversing `second` gives `5 -> 4`. Merging alternately: take `1` from `first`, then `5` from the reversed second, then `2`, then `4`, then the last remaining node `3` — giving `1 -> 5 -> 2 -> 4 -> 3`, matching the expected output.

Brute force

Copy all nodes into a list (array), then relink using two pointers walking inward from both ends of the array.

```
public void reorderListBrute(ListNode head) {
    if (head == null) return;
    List<ListNode> nodes = new ArrayList<>();
    for (ListNode node = head; node != null; node = node.next) {
        nodes.add(node);
    }
    int left = 0, right = nodes.size() - 1;
    while (left < right) {
        nodes.get(left).next = nodes.get(right);
        left++;
        if (left == right) break;
        nodes.get(right).next = nodes.get(left);
        right--;
    }
    nodes.get(left).next = null;   // terminate the reordered list
}
```

Optimized (find middle, reverse second half, merge alternately)

Three linear passes, O(1) extra space.

```
public void reorderList(ListNode head) {
    if (head == null || head.next == null) return;

    // 1) find the middle
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // 2) split into first half and second half
    ListNode second = slow.next;
    slow.next = null;

    // 3) reverse the second half
    ListNode prev = null;
    while (second != null) {
        ListNode next = second.next;
        second.next = prev;
        prev = second;
        second = next;
    }
    second = prev; // head of the reversed second half

    // 4) merge the two halves alternately
    ListNode first = head;
    while (second != null) {
        ListNode firstNext = first.next;
        ListNode secondNext = second.next;
        first.next = second;          // splice in the second-half node
        if (firstNext == null) break; // odd length: second-half tail already correct
        second.next = firstNext;      // hop back to continue the first half
        first = firstNext;
        second = secondNext;
    }
}
```

Version	Time	Space
Copy to array, relink	O(n)	O(n)
Fast/slow + reverse + merge	O(n)	O(1)

3.5 Swap Nodes in Pairs (LC 24)

Problem. Given the `head` of a singly linked list, swap every two adjacent nodes and return the head of the modified list. You must solve it by only changing node links (not the node values), and you may not use recursion if you want the O(1)-space guarantee (though a recursive solution is also acceptable and shown below for comparison).

- **Constraints:** `0 <= list length <= 100`; node values are typical integers; if the list has an odd number of nodes, the last node is left as-is.
- **Clarifying assumptions:** an empty list or a single-node list returns unchanged (there's nothing to swap); swaps are done pairwise from the front — `(1,2)`, `(3,4)`, … — not a full reversal of the whole list.

- **Why it's tricky:** swapping two nodes requires re-pointing **three** links correctly (the node before the pair, and the two nodes within the pair) in the right order, or you lose track of the rest of the list — a **dummy head** node avoids special-casing the very first swap, which would otherwise need to update the caller's `head` reference directly.

Example.

```
Input:  1 -> 2 -> 3 -> 4
Output: 2 -> 1 -> 4 -> 3
```

Intuition (diagram). Keep a `prev` pointer sitting just before each pair. For the pair `(first, second)`, re-point `first.next` past `second` to whatever `second` used to point to, then point `second.next` back at `first`, then have `prev` point at `second` (the new head of this pair). Advance `prev` to `first` (now the tail of this swapped pair) and repeat.

```
before:  dummy -> [1] -> [2] -> [3] -> [4] -> null
           ^prev   ^first ^second

relink:  first.next  = second.next   ([1] -> [3])
         second.next = first          ([2] -> [1])
         prev.next   = second         (dummy -> [2])

after pair 1:  dummy -> [2] -> [1] -> [3] -> [4] -> null
                                ^prev (advance to old 'first')

repeat for the next pair (first=[3], second=[4]):
after pair 2:  dummy -> [2] -> [1] -> [4] -> [3] -> null
```

Approach (step by step)

Key idea: a dummy node placed before `head` lets `prev` always be "the last correctly linked node so far," including before the very first swap — so the first pair doesn't need special-case code to update an external `head` variable.

1. Create `dummy` with `dummy.next = head`; set `prev = dummy`.
2. While there are at least two more nodes (`prev.next != null && prev.next.next != null`): a. Let `first = prev.next` and `second = first.next`. b. Relink: `first.next = second.next` (skip `first` past the pair), `second.next = first` (point the new pair head back at `first`), `prev.next = second` (attach the pair to the already-processed part of the list). c. Advance `prev = first` (now the tail of the just-swapped pair, ready to precede the next pair).
3. Return `dummy.next`, the new head of the list.

Worked trace on `1 -> 2 -> 3 -> 4`: the first pair `(1,2)` becomes `2 -> 1`, spliced in after `dummy`, giving `dummy -> 2 -> 1 -> 3 -> 4`; `prev` advances to node `1`. The second pair `(3,4)` becomes `4 -> 3`, spliced in after node `1`, giving `dummy -> 2 -> 1 -> 4 -> 3`. The loop then stops (`prev.next.next` is `null`). Returning `dummy.next` gives `2 -> 1 -> 4 -> 3`, matching the expected output.

Brute force

Collect node values into an array, swap adjacent pairs of values, and write them back into the existing nodes (touches values, which real interviews typically disallow, but useful as a warm-up correctness check).

```
public ListNode swapPairsBrute(ListNode head) {
    List<ListNode> nodes = new ArrayList<>();
    for (ListNode node = head; node != null; node = node.next) {
        nodes.add(node);
    }
    for (int i = 0; i + 1 < nodes.size(); i += 2) {
        int tmp = nodes.get(i).val;
        nodes.get(i).val = nodes.get(i + 1).val;
        nodes.get(i + 1).val = tmp;
    }
    return head;
}
```

Optimized (dummy head + iterative pointer relinking)

Rewires links directly; no value copying.

```
public ListNode swapPairs(ListNode head) {
    ListNode dummy = new ListNode(0, head);
    ListNode prev = dummy;

    while (prev.next != null && prev.next.next != null) {
        ListNode first = prev.next;
        ListNode second = first.next;

        first.next = second.next;   // first now points past the pair
        second.next = first;        // second becomes the pair's new head, pointing at firs
        prev.next = second;         // attach the swapped pair to the processed list

        prev = first;               // first is now the tail of this pair
    }
    return dummy.next;
}
```

Version	Time	Space
Swap values via array	O(n)	O(n)
Dummy head + relinking	O(n)	O(1)

---

4. Trees (new)

The core idea. Trees have no cycles, so almost every tree problem reduces to one of two moves: **post-order DFS** (gather answers from both children, *then* combine them at the current node — the natural fit when the answer depends on subtree results, like a height or a diameter), or **BFS with an explicit level boundary** (process one queue-snapshot's worth of nodes at a time — the natural fit when the answer is phrased "per level," like a right-side view or a zigzag order).

Reach for it when: the problem asks for the **longest path / deepest combination of two subtrees** (→ post-order DFS returning a value per node, with a side-channel max), for **every root-to-leaf path matching a condition** (→ DFS + backtracking over a running path), or for **anything phrased "per level"** — visible node, alternating order, or level width (→ BFS with `queue.size()` snapshotted before draining it).

Assume this node definition throughout:

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

4.1 Diameter of Binary Tree (LC 543)

Problem. Given the root of a binary tree, return the length of its **diameter**: the number of **edges** on the longest path between any two nodes in the tree. That path does not have to pass through the root.

- **Input:** the `root` of a binary tree (may be `null`).
- **Output:** an `int`, the edge count of the longest path between any two nodes.
- **Constraints:** up to `10^4` nodes; node values fit in a 32-bit int and are irrelevant to the answer (only shape matters).
- **Clarifying assumptions:** confirm the answer is counted in **edges, not nodes** (a single-node tree has diameter 0, a two-node tree has diameter 1); confirm the two endpoints of the longest path need not include the root.
- **Why it's tricky:** the longest path in the whole tree doesn't have to pass through the root, so you can't just compute `leftHeight + rightHeight` once at the top — the true diameter might be entirely inside one subtree. The fix is to compute that sum **at every node** during a single post-order pass and keep a running max, rather than trying to derive the diameter from a top-level formula.

Example.

```
Input:  root = [1, 2, 3, 4, 5]
Output: 3          (longest path: 4 -> 2 -> 1 -> 3, or 5 -> 2 -> 1 -> 3, each 3 edges)
```

Intuition (diagram). For any node, the longest path that passes **through** it is `(height of its left subtree) + (height of its right subtree)` — walk all the way down the left side, cross the node, walk all the way down the right side. The overall diameter is the best such sum over **every** node, not just the root, because the longest path might be tucked entirely inside one subtree and never touch the root at all.

```
        1
      /   \
    2       3
  /   \
4       5

height(4) = 1, height(5) = 1
at node 2: leftH=1, rightH=1  -> path through 2 = 1+1 = 2 edges (4-2-5)
height(2) = 1 + max(1,1) = 2
at node 3: leaf -> path through 3 = 0
at node 1: leftH=height(2)=2, rightH=height(3)=1
           -> path through 1 = 2+1 = 3 edges (4-2-1-3, or 5-2-1-3)
diameter = max(2, 0, 3) = 3
```

Approach (step by step)

Key idea / invariant: run a single post-order DFS that returns each node's **height** (number of nodes on the longest downward path, or equivalently `0` for `null`). While computing that height, also update a **global (or instance) max** with `leftHeight + rightHeight` — the edge count of the longest path that bends at this exact node. Because every node in the tree gets this check exactly once, the running max ends up holding the best path over the *whole* tree, not just paths through the root.

1. Define a helper `height(node)` that returns `0` for `null` (base case).
2. Recurse: `leftHeight = height(node.left)`, `rightHeight = height(node.right)`.
3. **Update the answer** *before* returning: `diameter = max(diameter, leftHeight + rightHeight)` — this is the longest path that uses `node` as its highest (bending) point.
4. Return `1 + max(leftHeight, rightHeight)` — this node's own height, for its parent's step 3.
5. After the DFS finishes at the root, `diameter` holds the answer.

Worked trace on `[1, 2, 3, 4, 5]` (post-order: 4, 5, 2, 3, 1): | call | leftHeight | rightHeight | diameter update | returns (height) | |---|---|---|---|---| | height(4) | 0 | 0 | max(0, 0+0) = 0 | 1 | | height(5) | 0 | 0 | max(0, 0+0) = 0 | 1 | | height(2) | 1 | 1 | max(0, 1+1) = 2 | 2 | | height(3) | 0 | 0 | max(2, 0+0) = 2 | 1 | | height(1) | 2 | 1 | max(2, 2+1) = **3** | 3 |

Final `diameter = 3`, matching the expected output.

Brute force

For every node, independently compute the height of its left and right subtree from scratch, and take the max of `leftHeight + rightHeight` over all nodes — recomputing height recursively each time is redundant work.

```
public int diameterOfBinaryTreeBrute(TreeNode root) {
    if (root == null) return 0;
    int throughRoot = height(root.left) + height(root.right);
    int bestInLeft = diameterOfBinaryTreeBrute(root.left);
    int bestInRight = diameterOfBinaryTreeBrute(root.right);
    return Math.max(throughRoot, Math.max(bestInLeft, bestInRight));
}

private int height(TreeNode node) {
    if (node == null) return 0;
    return 1 + Math.max(height(node.left), height(node.right));   // recomputed from scratc
}
```

Optimized (single post-order pass)

Compute height and update the diameter candidate in the same recursive call.

```
private int diameter = 0;

public int diameterOfBinaryTree(TreeNode root) {
    diameter = 0;
    height(root);
    return diameter;
}

private int height(TreeNode node) {
    if (node == null) return 0;
    int leftHeight = height(node.left);
    int rightHeight = height(node.right);
    diameter = Math.max(diameter, leftHeight + rightHeight);   // longest path bending at t
    return 1 + Math.max(leftHeight, rightHeight);
}
```

Version	Time	Space
Brute force (recompute height per node)	O(n^2) worst case	O(h) call stack
Single post-order pass	O(n)	O(h) call stack

4.2 Path Sum II (LC 113)

Problem. Given the root of a binary tree and an integer `targetSum`, return **all** root-to-leaf paths where each path's node values sum to `targetSum`. Each path should be returned as a list of the values along it, root first.

- **Input:** the `root` of a binary tree (may be `null`), and an int `targetSum`.
- **Output:** a `List<List<Integer>>`, one inner list per qualifying root-to-leaf path.
- **Constraints:** up to `5000` nodes; node values are 32-bit ints, typically `-1000` to `1000`; `-1000 <= targetSum <= 1000`.
- **Clarifying assumptions:** confirm a "leaf" means a node with no children (a path must run all the way to a leaf — an internal node hitting `targetSum` early doesn't count); confirm negative values are possible, so you can't prune purely on "sum already exceeds target."

- **Why it's tricky:** you need to explore **every** root-to-leaf path (not stop at the first match), and you're building one shared mutable `path` list across the whole DFS — which means you must **backtrack** (remove the node you just added) after recursing into its children, or every subsequent path will incorrectly keep leftover entries from branches that have nothing to do with it.

Example.

```
Input:  root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: [[5,4,11,2], [5,8,4,5]]
```

Intuition (diagram). Walk every root-to-leaf path while carrying a running list of the values seen so far and the running sum. When you reach a leaf, check if the running sum equals the target; if so, snapshot the current path into the answer. Whether it matched or not, **remove** the node from the running path before returning to the parent — otherwise the path list leaks into sibling branches that never visited this node.

```
                5
              /     \
            4         8
          /         /    \
        11        13       4
      /   \             /   \
     7     2           5     1

path 5-4-11-7: sum=27, not a leaf-match target(22) -> no
path 5-4-11-2: sum=22, leaf, matches!            -> record [5,4,11,2]
path 5-8-13:   sum=26, leaf, no match
path 5-8-4-5:  sum=22, leaf, matches!            -> record [5,8,4,5]
path 5-8-4-1:  sum=18, leaf, no match
```

Approach (step by step)

Key idea / invariant: carry one mutable `path` list and a running `sum` down the recursion. At every call, the invariant is: *`path` contains exactly the values from the root down to (and including) the current node.* Adding a node before recursing and removing it (backtracking) right after the recursive calls return is what keeps that invariant true for every sibling branch, not just the first one explored.

1. Base case: if `node == null`, return immediately (nothing to add).
2. Add `node.val` to `path` and add it to the running `sum`.
3. If `node` is a **leaf** (`left == null && right == null`) and `sum == targetSum`, the current `path` is a valid answer — copy it into the result list (copy, because `path` keeps mutating).
4. Otherwise, recurse into `node.left`, then `node.right`, with the updated `path` / `sum`.
5. **Backtrack**: remove `node.val` from the end of `path` (and undo the sum) before returning — this restores the invariant for the caller's next sibling to explore.

Worked trace on the example (target 22), following the `5 -> 4 -> 11 -> ...` branch:

```
path=[5]         sum=5          (not leaf)
path=[5,4]       sum=9          (not leaf)
path=[5,4,11]    sum=20         (not leaf)
path=[5,4,11,7]  sum=27  leaf, 27 != 22 -> no match
  backtrack: path=[5,4,11]      sum=20
path=[5,4,11,2]  sum=22  leaf, 22 == 22 -> record [5,4,11,2]
  backtrack: path=[5,4,11] -> [5,4] -> [5] -> [] as the DFS unwinds back to root
```

Brute force

Collect every complete root-to-leaf path into a list first, then filter by sum afterward — correct, but does extra bookkeeping instead of checking as you go.

```
public List<List<Integer>> pathSumBrute(TreeNode root, int targetSum) {
    List<List<Integer>> allPaths = new ArrayList<>();
    collectAllPaths(root, new ArrayList<>(), allPaths);
    List<List<Integer>> result = new ArrayList<>();
    for (List<Integer> path : allPaths) {
        int sum = 0;
        for (int v : path) sum += v;
        if (sum == targetSum) result.add(path);
    }
    return result;
}

private void collectAllPaths(TreeNode node, List<Integer> path, List<List<Integer>> allPath
    if (node == null) return;
    path.add(node.val);
    if (node.left == null && node.right == null) {
        allPaths.add(new ArrayList<>(path));    // snapshot every leaf path, unfiltered
    } else {
        collectAllPaths(node.left, path, allPaths);
        collectAllPaths(node.right, path, allPaths);
    }
    path.remove(path.size() - 1);
}
```

Optimized (DFS with backtracking, check sum inline)

Track the running sum alongside the path so each leaf is checked in one pass.

```
public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
    List<List<Integer>> result = new ArrayList<>();
    dfs(root, targetSum, 0, new ArrayList<>(), result);
    return result;
}

private void dfs(TreeNode node, int targetSum, int sum, List<Integer> path,
                 List<List<Integer>> result) {
    if (node == null) return;
    path.add(node.val);                     // choose: add this node to the path
    sum += node.val;
    boolean isLeaf = node.left == null && node.right == null;
    if (isLeaf && sum == targetSum) {
        result.add(new ArrayList<>(path));   // snapshot a copy -- path keeps mutating
    }
    dfs(node.left, targetSum, sum, path, result);
    dfs(node.right, targetSum, sum, path, result);
    path.remove(path.size() - 1);           // un-choose: backtrack before returning
}
```

Version	Time	Space
Collect all paths, filter after	O(n^2) worst case (path copies)	O(n) paths stored
DFS + backtracking, check inline	O(n^2) worst case (matches copy their path)	O(h) call stack + O(h) path

---

4.3 Binary Tree Right Side View (LC 199)

Problem. Given the root of a binary tree, imagine standing on the **right** side of it — return the values of the nodes you can see, ordered from top to bottom.

- **Input:** the `root` of a binary tree (may be `null`).
- **Output:** a `List<Integer>`, one value per level — the rightmost visible node at that level.
- **Constraints:** up to `100` nodes (typical LeetCode bound, though the technique scales to larger trees); node values fit in a 32-bit int.
- **Clarifying assumptions:** confirm "rightmost visible" means the **last** node encountered in left-to-right order at each level (not necessarily a node with an actual right child — a level might end in a node that only has a *left* child, and that node is still the one visible from the right, since nothing sits to its right at that depth).
- **Why it's tricky:** the natural-looking shortcut "always go right first" only reaches nodes reachable via right-children, but a level can end in a node whose *last* right-hand branch happened to dead-end into just a left child — you need to reason about **whole levels**, not right-child chains, to get the rightmost node at every depth correctly.

Example.

```
Input:  root = [1, 2, 3, null, 5, null, 4]
Output: [1, 3, 4]
```

Intuition (diagram). BFS naturally visits nodes left to right within each level. The node you see from the right at any level is simply the **last** one processed in that level's pass — no special-casing needed, even

when that node only has a left child (like node `2`, whose only child `5` still ends up as the last node of level 2 down that branch).

```
      1              level 0: [1]              -> last = 1
    /   \
  2       3          level 1: [2, 3]            -> last = 3
    \       \
      5       4      level 2: [5, 4]            -> last = 4

right side view = [1, 3, 4]
```

Approach (step by step)

Key idea / invariant: at the top of every BFS iteration, the queue holds exactly one level's worth of nodes, left to right — the same snapshot-the-size trick used for level-order traversal. The **last** node dequeued while draining that snapshot is, by construction, the rightmost node at that depth, regardless of whether it got there via a left or right pointer.

1. If the tree is empty, return an empty list.
2. Push `root` into the queue.
3. While the queue isn't empty: a. Record `size = queue.size()` — the current level's node count. b. Poll exactly `size` nodes, enqueueing each one's non-null children as you go. c. The **last** node polled in this batch (`i == size - 1`) is this level's rightmost visible node — append its value to the result.
4. Return the result once the queue drains.

Worked trace on `[1, 2, 3, null, 5, null, 4]`:

```
queue=[1]        size=1  poll 1 (last)         -> result=[1]
queue=[2,3]      size=2  poll 2, poll 3 (last)  -> result=[1,3]
queue=[5,4]      size=2  poll 5, poll 4 (last)  -> result=[1,3,4]
queue=[]         loop ends
```

Brute force

Do a full level-order traversal (collecting every value per level), then take the last element of each level list.

```
public List<Integer> rightSideViewBrute(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    List<List<Integer>> levels = new ArrayList<>();
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        levels.add(level);
    }
    for (List<Integer> level : levels) {
        result.add(level.get(level.size() - 1));   // take the last of each fully-built lev
    }
    return result;
}
```

Optimized (BFS, keep only the last node per level)

Skip storing whole levels — just remember the value seen on the last poll of each pass.

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (i == size - 1) {
                result.add(node.val);              // last node polled this level = rightmo
            }
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return result;
}
```

Version	Time	Space
Collect full levels, take last	O(n)	O(n) for stored levels
BFS, keep only last per level	O(n)	O(w) queue (w = max width)

4.4 Binary Tree Zigzag Level Order Traversal (LC 103)

Problem. Given the root of a binary tree, return its level order traversal, but alternate the direction of each level: left-to-right, then right-to-left, then left-to-right, and so on.

- **Input:** the `root` of a binary tree (may be `null`).
- **Output:** a `List<List<Integer>>`, one inner list per level, with alternating order.
- **Constraints:** up to `2000` nodes; node values fit in a 32-bit int and may repeat.
- **Clarifying assumptions:** confirm level 0 (the root's level) goes **left-to-right** (the first flip happens at level 1); confirm an empty tree returns an empty list.
- **Why it's tricky:** BFS discovers each level in left-to-right order regardless of what you want to *output* — you can't change how the queue discovers children (that would break the traversal), so the flip has to happen only when you **write the level's values into the output list**, alternating between appending and prepending (or building normally and reversing every other level).

Example.

```
Input:  root = [3, 9, 20, null, null, 15, 7]
Output: [[3], [20, 9], [15, 7]]
```

Intuition (diagram). BFS always *discovers* nodes left to right within a level — that part never changes. The zigzag only affects the order values are **recorded** into each level's output list: even levels (0, 2, 4, ...) record left-to-right as usual; odd levels (1, 3, 5, ...) record right-to-left, which you can get by reversing the level's list (or by inserting each value at the front instead of the back).

```
      3                level 0 (L->R): discover [3]          -> output [3]
    /   \
  9       20            level 1 (R->L): discover [9, 20]      -> output [20, 9]
         /   \
        15    7         level 2 (L->R): discover [15, 7]      -> output [15, 7]
```

Approach (step by step)

Key idea / invariant: run ordinary BFS with the standard `queue.size()` level-boundary trick; the *discovery* order within a level is always left-to-right because children are always enqueued left-then-right. Maintain a boolean flag `leftToRight` that flips after every level; use it only to decide how to **write** the level's values (append normally, or reverse before adding to the result — equivalently, insert into a `Deque` from the front when going right-to-left).

1. If the tree is empty, return an empty list.
2. Push `root` into the queue; set `leftToRight = true`.
3. While the queue isn't empty: a. Record `size = queue.size()`. b. Poll `size` nodes in the usual left-to-right discovery order, enqueueing children as usual, and collect their values into a temporary `level` list (always append here, in discovery order). c. If `leftToRight` is `false`, reverse `level` before adding it to the result. d. Flip `leftToRight`.
4. Return the result once the queue drains.

Worked trace on `[3, 9, 20, null, null, 15, 7]`:

```
leftToRight=true   queue=[3]      discover [3]        -> keep order  -> result=[[3]]
leftToRight=false  queue=[9,20]   discover [9,20]      -> reverse     -> result=[[3],[20,9]
leftToRight=true   queue=[15,7]   discover [15,7]      -> keep order  -> result=[[3],[20,9]
```

Brute force

Run a normal level-order BFS first, then reverse every other level afterward as a post-processing pass.

```
public List<List<Integer>> zigzagLevelOrderBrute(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    for (int i = 1; i < result.size(); i += 2) {
        Collections.reverse(result.get(i));    // fix up odd levels after the fact
    }
    return result;
}
```

Optimized (BFS with a direction flag)

Reverse the level in place only when the direction calls for it — no separate pass.

```
public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    boolean leftToRight = true;
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);                    // always discover left to right
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        if (!leftToRight) {
            Collections.reverse(level);              // flip only how we *record* this lev
        }
        result.add(level);
        leftToRight = !leftToRight;
    }
    return result;
}
```

Version	Time	Space
BFS, then reverse odd levels after	O(n)	O(w) queue + O(n) output
BFS with direction flag	O(n)	O(w) queue + O(n) output

4.5 Maximum Width of Binary Tree (LC 662)

Problem. Given the root of a binary tree, return its **maximum width**: the largest of the widths across all levels, where a level's width is the number of slots between its leftmost and rightmost **non-null** nodes (inclusive), counting the null slots in between as if the tree were a complete binary tree.

- **Input:** the `root` of a binary tree (may be `null`).
- **Output:** an `int`, the maximum width over all levels.
- **Constraints:** up to `3000` nodes; the answer fits in a 32-bit signed int (per LeetCode); node values fit in a 32-bit int.
- **Clarifying assumptions:** confirm width counts the **gap** between leftmost and rightmost non-null nodes (including missing/null positions between them), not just the count of actual non-null nodes at that level; confirm a single-node tree has width 1.
- **Why it's tricky:** you need a way to know *how far apart* two nodes at the same level would be if the tree were laid out as a complete binary tree, without actually materializing all the null placeholders. The trick is borrowing the classic **complete-binary-tree indexing** scheme (root = index `1`, left child of `i` = `2i`, right child of `i` = `2i+1`) and computing each level's width as `lastIndex - firstIndex + 1` — and then guarding against those indices overflowing for deep, lopsided trees.

Example.

```
Input:  root = [1, 3, 2, 5, 3, null, 9]
Output: 4
```

Intuition (diagram). Give every node the index it *would* have in a complete binary tree (root = 1, left child of index `i` = `2i`, right child = `2i+1`) — this index encodes exactly where the node sits, including the gaps left by missing siblings. A level's width is then just `lastIndex - firstIndex + 1` for the non-null nodes actually present at that level.

```
              1
            i=1
          /      \
        3          2
       i=2        i=3
      /   \           \
     5     3           9
    i=4   i=5         i=7          (index 6 is missing -- node 2's left child is null)

level with indices {4, 5, 7}: width = 7 - 4 + 1 = 4   -> this is the max
```

Approach (step by step)

Key idea / invariant: run BFS, but every queue entry carries a `(node, index)` pair instead of just the node, using the complete-binary-tree indexing rule: a node at index `i` has left child at index `2i` and right child at `2i + 1`. At the top of each level, the **first** index in the queue's snapshot and the **last** index give the width directly: `lastIndex − firstIndex + 1`.

1. If the tree is empty, return `0`.
2. Push `(root, 1)` into the queue (root gets index 1).
3. While the queue isn't empty: a. Record `size = queue.size()` for this level's snapshot. b. Read the index of the **first** entry in the snapshot before polling anything — call it `first`. c. Poll `size` entries. For each `(node, index)`: enqueue `(node.left, 2*index)` and `(node.right, 2*index + 1)` when those children exist. Track the index of the **last** entry polled — call it `last`. d. This level's width is `last − first + 1`; update the running max.
4. Return the max width seen across all levels.

Overflow note: for a deep, heavily left- or right-skewed tree, raw indices `2i` / `2i+1` can grow exponentially with depth and overflow a 32-bit (or even 64-bit) integer long before the tree itself gets large. The standard fix is to **re-base indices at the start of every level**: subtract `first` (the level's own first index) from every index before using it to compute that level's children indices, so each level's numbers restart near zero instead of compounding across all previous levels.

Worked trace on `[1, 3, 2, 5, 3, null, 9]`:

```
level 0: queue=[(1,i=1)]                          first=1, last=1  -> width=1
level 1: queue=[(3,i=2),(2,i=3)]                  first=2, last=3  -> width=2
level 2: queue=[(5,i=4),(3,i=5),(9,i=7)]          first=4, last=7  -> width = 7−4+1 = 4
max width = 4
```

Brute force

Do a full level-order BFS while also tracking each node's complete-binary-tree index, but recompute indices without any re-basing — correct for small/shallow trees, risks overflow on deep skewed ones since indices are used as plain `int`.

```
public int widthOfBinaryTreeBrute(TreeNode root) {
    if (root == null) return 0;
    Queue<TreeNode> nodeQueue = new LinkedList<>();
    Queue<Integer> indexQueue = new LinkedList<>();
    nodeQueue.offer(root);
    indexQueue.offer(1);
    int maxWidth = 0;
    while (!nodeQueue.isEmpty()) {
        int size = nodeQueue.size();
        int first = indexQueue.peek();
        int last = first;
        for (int i = 0; i < size; i++) {
            TreeNode node = nodeQueue.poll();
            int index = indexQueue.poll();
            last = index;                               // raw index, can overflow if deep +
            if (node.left != null) {
                nodeQueue.offer(node.left);
                indexQueue.offer(2 * index);
            }
            if (node.right != null) {
                nodeQueue.offer(node.right);
                indexQueue.offer(2 * index + 1);
            }
        }
        maxWidth = Math.max(maxWidth, last - first + 1);
    }
    return maxWidth;
}
```

Optimized (BFS with per-level index re-basing)

Subtract each level's own first index before computing children indices, keeping numbers small regardless of tree depth.

```
public int widthOfBinaryTree(TreeNode root) {
    if (root == null) return 0;
    Queue<TreeNode> nodeQueue = new LinkedList<>();
    Queue<Integer> indexQueue = new LinkedList<>();
    nodeQueue.offer(root);
    indexQueue.offer(0);                                  // re-based: root is always index 0
    int maxWidth = 0;
    while (!nodeQueue.isEmpty()) {
        int size = nodeQueue.size();
        int first = indexQueue.peek();
        int last = first;
        for (int i = 0; i < size; i++) {
            TreeNode node = nodeQueue.poll();
            int index = indexQueue.poll() - first;        // re-base relative to this level's
            last = index + first;
            if (node.left != null) {
                nodeQueue.offer(node.left);
                indexQueue.offer(2 * index);
            }
            if (node.right != null) {
                nodeQueue.offer(node.right);
                indexQueue.offer(2 * index + 1);
            }
        }
        maxWidth = Math.max(maxWidth, last - first + 1);
    }
    return maxWidth;
}
```

Version	Time	Space
BFS with raw complete-tree indices	O(n)	O(w) queue (risk of index overflow if deep + skewed)
BFS with per-level re-based indices	O(n)	O(w) queue

---

5. Grid & Graph DFS (new)

The core idea. A 2D grid is just a graph where every cell has up to 4 neighbors, and many grid problems boil down to a **multi-source flood fill** rather than one DFS per starting point: seed the search from every border cell (or every cell that already satisfies some property) at once, and let connectivity do the rest. On general graphs, the same DFS toolkit answers structural questions like "is this a tree?" — often faster with **union-find**, which tracks connected components without ever building an explicit adjacency list.

Reach for it when: you see "regions surrounded by X", "reaches both/all edges", "flows downhill/uphill", "capture/flip cells not connected to the border", or "given n nodes and a list of edges, is this a valid tree / how many connected components".

5.1 Surrounded Regions (LC 130)

Problem. Given an `m x n` matrix `board` containing only the characters `'X'` and `'O'`, **capture** all regions of `'O'` that are **fully surrounded** by `'X'` on all sides, by flipping every `'O'` in such a region to `'X'`. A region is a maximal set of `'O'`s connected 4-directionally (up/down/left/right). A region is **not** captured if any of its cells touches the border of the board (row 0, last row, column 0, or last column) — such a region is considered "connected to the outside" and stays `'O'`. Modify `board` in place; return nothing.

- **Constraints:** `1 <= rows, cols <= 200`; `board[i][j]` is `'X'` or `'O'`.
- **Clarifying assumptions:** connectivity is 4-directional, not 8-directional (no diagonals); a cell that is itself on the border is automatically safe even if it has no `'O'` neighbors; the board is modified in place, not returned as a new matrix.
- **Why it's tricky:** the natural instinct is to check, for every `'O'` region, whether it touches the border — but that means re-deriving "does this region touch the border" from scratch for every region. It's far cheaper to flip the problem around: find everything connected to the border first (in one multi-source search), and treat everything else as captured.

Example.

```
Input:
X X X X
X O O X
X X O X
X O X X

Output:
X X X X
X X X X
X X X X
X O X X
```

Intuition (diagram). Start the search from the border, not from the interior. Any `'O'` reachable from a border `'O'` is safe forever; anything left unmarked after that search is, by definition, surrounded, and

gets flipped. Below, `(3,1)` is the only `'O'` sitting on the border, so a DFS from it is the entire "safe" search — it has no `'O'` neighbors, so the search stops immediately, leaving it as the sole survivor.

```
board (before):            safe cells found by
                            DFS from border 'O's:
row0: X X X X               row0: .  .  .  .
row1: X O O X               row1: .  .  .  .
row2: X X O X               row2: .  .  .  .
row3: X O X X               row3: .  S  .  .

board (after flipping every unmarked 'O' to 'X'):
row0: X X X X
row1: X X X X
row2: X X X X
row3: X O X X
```

Approach (step by step)

Key idea: flip the question from "does this interior region touch the border?" (checked once per region) to "what's reachable from the border?" (checked once, globally). Everything reachable from a border `'O'` via `'O'`-to-`'O'` moves is safe; everything else is surrounded.

1. Scan the four edges of `board` (row 0, last row, column 0, last column). For every `'O'` found there, run a DFS/BFS that marks it, and every `'O'` reachable from it through 4-way `'O'`-to-`'O'` moves, as **safe** in a parallel `boolean[][] safe` grid.
2. Once every border `'O'` has seeded its search, `safe` contains exactly the cells that belong to a region touching the border.
3. Make a second pass over the whole board: any cell that is `'O'` but **not** marked `safe` is surrounded — flip it to `'X'`.

Why start from the border (the inverse trick): checking "can region R reach the border?" for every region separately means redoing a traversal per region. Checking "what can the border reach?" does the same connectivity work exactly once, because reachability from the border is transitive over a whole component — if one cell of a component touches the border, the entire component is safe, and a single seeded DFS discovers all of it at once.

Worked trace on the example board: - Scan the border: the only border cell holding `'O'` is `(3,1)`. - DFS from `(3,1)`: mark it safe. Its neighbors are `(2,1)='X'`, `(3,0)='X'`, `(3,2)='X'` — no `'O'` neighbors, so the DFS from this seed ends immediately. - No other border cell is `'O'`, so the safe search is done. `safe` = `{(3,1)}`. - Second pass: `(1,1)`, `(1,2)`, `(2,2)` are `'O'` but not in `safe` → flip all three to `'X'`. `(3,1)` is `'O'` and in `safe` → left alone. - Result matches the expected output: only `(3,1)` remains `'O'`.

Brute force

For every `'O'` cell independently, DFS outward and check if that cell's own component reaches the border; if not, flip just that cell. Correct but re-walks the same component's reachability from scratch for every cell inside it.

```
public void solveBrute(char[][] board) {
    if (board == null || board.length == 0) return;
    int rows = board.length, cols = board[0].length;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (board[r][c] == 'O' && !reachesBorder(board, r, c, new boolean[rows][cols]))
                board[r][c] = 'X';   // this cell's component can't reach the border
            }
        }
    }
}

private boolean reachesBorder(char[][] board, int r, int c, boolean[][] visited) {
    int rows = board.length, cols = board[0].length;
    if (r < 0 || r >= rows || c < 0 || c >= cols) return false;
    if (visited[r][c] || board[r][c] != 'O') return false;
    if (r == 0 || r == rows - 1 || c == 0 || c == cols - 1) return true;
    visited[r][c] = true;
    return reachesBorder(board, r - 1, c, visited)
        || reachesBorder(board, r + 1, c, visited)
        || reachesBorder(board, r, c - 1, visited)
        || reachesBorder(board, r, c + 1, visited);
}
```

Optimized (multi-source DFS from the border, then flip the rest)

Seed the search from every border `'O'` once, then flip whatever was never reached.

```
public void solve(char[][] board) {
    if (board == null || board.length == 0) return;
    int rows = board.length, cols = board[0].length;
    boolean[][] safe = new boolean[rows][cols];

    for (int c = 0; c < cols; c++) {
        markSafe(board, 0, c, safe);
        markSafe(board, rows - 1, c, safe);
    }
    for (int r = 0; r < rows; r++) {
        markSafe(board, r, 0, safe);
        markSafe(board, r, cols - 1, safe);
    }

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (board[r][c] == 'O' && !safe[r][c]) {
                board[r][c] = 'X';   // never reached from the border -> surrounded
            }
        }
    }
}

private void markSafe(char[][] board, int r, int c, boolean[][] safe) {
    int rows = board.length, cols = board[0].length;
    if (r < 0 || r >= rows || c < 0 || c >= cols) return;
    if (safe[r][c] || board[r][c] != 'O') return;
    safe[r][c] = true;              // reachable from the border, never flip this
    markSafe(board, r - 1, c, safe);
    markSafe(board, r + 1, c, safe);
    markSafe(board, r, c - 1, safe);
    markSafe(board, r, c + 1, safe);
}
```

Version	Time	Space
Per-cell reachability check	O((rows·cols)^2) worst case	O(rows·cols)
Multi-source DFS from border	O(rows·cols)	O(rows·cols)

5.2 Pacific Atlantic Water Flow (LC 417)

Problem. Given an `m x n` integer matrix `heights` where `heights[r][c]` is the height of the cell at `(r, c)`, the Pacific Ocean touches the **top row and left column**, and the Atlantic Ocean touches the **bottom row and right column**. Water can flow from a cell to an adjacent (4-directional) cell only if the adjacent cell's height is **less than or equal to** the current cell's height (water flows downhill or across flat ground). Return the list of coordinates `[r, c]` from which water can flow to **both** oceans.

- **Constraints:** `1 <= rows, cols <= 200`; `0 <= heights[r][c] <= 10^5`.
- **Clarifying assumptions:** flow is allowed onto equal-height cells, not just strictly lower ones; every border cell trivially can flow into its adjacent ocean(s); order of the returned coordinates doesn't matter.
- **Why it's tricky:** the natural simulation — from every cell, DFS forward following "downhill or equal" and see if you hit each border — checks `rows*cols` starting cells, each doing up to

`rows*cols` work. The fix requires **reversing the direction of the search**: instead of asking "can I flow down to the ocean from here?", ask "which cells could have flowed down to me?" by walking *uphill-or-equal* from the ocean's edge inward.

Example.

```
Input:
heights =
1 3 2
2 4 3
1 2 1

Output: [(0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1)]
(every cell except the low corners (0,0) and (2,2))
```

Intuition (diagram). Run one search from the whole Pacific border (top row + left column) and one from the whole Atlantic border (bottom row + right column). In each search, step to a neighbor only if the neighbor's height is **greater than or equal to** the current cell's height — that's the reverse of "water flows downhill", so walking uphill-or-equal from an ocean's edge reconstructs exactly the set of cells whose water could have flowed down into that edge. Intersect the two reachable sets for the answer.

```
heights:                Pacific-reachable (P):     Atlantic-reachable (A):
row0: 1 3 2              row0: P P P                row0: . A A
row1: 2 4 3              row1: P P P                row1: A A A
row2: 1 2 1              row2: P P .                row2: A A A

intersection (both oceans, '*' = answer cell):
row0: . * *
row1: * * *
row2: * * .
```

Approach (step by step)

Key idea: don't simulate water flowing forward from every one of the `rows*cols` cells. Instead run the search **backwards, twice** — once seeded from all Pacific border cells, once from all Atlantic border cells — walking to a neighbor whenever `heights[neighbor] >= heights[current]`. Each of those two searches is a single multi-source DFS/BFS, so the whole algorithm is two O(rows·cols) passes instead of `rows*cols` separate forward searches.

1. Build two boolean grids, `pacific` and `atlantic`, both initially all `false`.
2. Seed a DFS from every cell in row 0 and column 0 into `pacific`; seed a DFS from every cell in the last row and last column into `atlantic`.
3. In each DFS, move from the current cell to a neighbor only if that neighbor hasn't been visited yet **and** `heights[neighbor] >= heights[current]` (uphill-or-flat, the reverse of the forward flow rule).
4. After both searches finish, a cell `(r, c)` can reach both oceans exactly when `pacific[r][c] && atlantic[r][c]` — collect all such cells.

Worked trace on the example grid (`heights` above): - **Pacific search** seeds at row 0 (`(0,0)=1`, `(0,1)=3`, `(0,2)=2`) and column 0 (`(0,0)`, `(1,0)=2`, `(2,0)=1`). From `(0,0)=1` it climbs to `(0,1)=3` and `(1,0)=2` (both ≥ 1); from `(0,1)=3` it climbs to `(1,1)=4`; from `(2,0)=1` it climbs to `(2,1)=2`; from

`(0,2)=2` it climbs to `(1,2)=3`. It **cannot** step from `(0,1)=3` to `(0,2)=2` (2 < 3) or from `(2,1)=2` to `(2,2)=1` (1 < 2). Final Pacific set: every cell except `(2,2)`. - **Atlantic search** seeds at row 2 and column 2 symmetrically, and similarly cannot climb into `(0,0)=1` from either `(0,1)=3` or `(1,0)=2` (1 < both). Final Atlantic set: every cell except `(0,0)`. - Intersection: every cell except `(0,0)` and `(2,2)` — the two low corners that each can only reach the ocean they're already sitting on. Matches the expected output.

Brute force

From every single cell, DFS **forward** (downhill-or-equal) and check whether that search ever touches the Pacific border, then repeat to check the Atlantic border.

```
public List<List<Integer>> pacificAtlanticBrute(int[][] heights) {
    List<List<Integer>> result = new ArrayList<>();
    if (heights == null || heights.length == 0 || heights[0].length == 0) return result;
    int rows = heights.length, cols = heights[0].length;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            boolean toPacific = canReach(heights, r, c, true, new boolean[rows][cols], heig
            boolean toAtlantic = canReach(heights, r, c, false, new boolean[rows][cols], he
            if (toPacific && toAtlantic) result.add(Arrays.asList(r, c));
        }
    }
    return result;
}

private boolean canReach(int[][] heights, int r, int c, boolean pacific, boolean[][] visite
    int rows = heights.length, cols = heights[0].length;
    if (r < 0 || r >= rows || c < 0 || c >= cols) return false;
    if (visited[r][c] || heights[r][c] > fromHeight) return false;   // must flow downhill
    if (pacific && (r == 0 || c == 0)) return true;
    if (!pacific && (r == rows - 1 || c == cols - 1)) return true;
    visited[r][c] = true;
    int h = heights[r][c];
    return canReach(heights, r - 1, c, pacific, visited, h)
        || canReach(heights, r + 1, c, pacific, visited, h)
        || canReach(heights, r, c - 1, pacific, visited, h)
        || canReach(heights, r, c + 1, pacific, visited, h);
}
```

Optimized (reverse multi-source DFS from each ocean)

Walk uphill-or-flat inward from each ocean's border, once each; intersect the results.

```
public List<List<Integer>> pacificAtlantic(int[][] heights) {
    List<List<Integer>> result = new ArrayList<>();
    if (heights == null || heights.length == 0 || heights[0].length == 0) return result;
    int rows = heights.length, cols = heights[0].length;
    boolean[][] pacific = new boolean[rows][cols];
    boolean[][] atlantic = new boolean[rows][cols];

    for (int c = 0; c < cols; c++) {
        dfs(heights, 0, c, pacific);
        dfs(heights, rows - 1, c, atlantic);
    }
    for (int r = 0; r < rows; r++) {
        dfs(heights, r, 0, pacific);
        dfs(heights, r, cols - 1, atlantic);
    }

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (pacific[r][c] && atlantic[r][c]) {
                result.add(Arrays.asList(r, c));
            }
        }
    }
    return result;
}

private void dfs(int[][] heights, int r, int c, boolean[][] reachable) {
    reachable[r][c] = true;
    int rows = heights.length, cols = heights[0].length;
    int[][] dirs = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
        if (reachable[nr][nc]) continue;
        if (heights[nr][nc] < heights[r][c]) continue;   // reverse flow: only step uphill
        dfs(heights, nr, nc, reachable);
    }
}
```

Version	Time	Space
Forward DFS from every cell	O((rows·cols)^2)	O(rows·cols)
Reverse DFS from each ocean	O(rows·cols)	O(rows·cols)

5.3 Graph Valid Tree (LC 261)

Problem. Given `n` nodes labeled `0` to `n - 1` and a list of undirected `edges` (each `edges[i] = [a, b]`), determine whether these edges form a **valid tree** — a connected graph with no cycles.

- **Constraints:** `1 <= n <= 2000`; `0 <= edges.length <= 5000`; no self-loops (`a != b`) and no duplicate edges in the input.
- **Clarifying assumptions:** the graph is undirected; nodes not mentioned in any edge are still part of the `n` nodes and must end up connected for the answer to be `true`; `n == 1` with zero edges is a valid (trivial, single-node) tree.

- **Why it's tricky:** a valid tree needs **two** independent conditions to both hold — exactly `n - 1` edges, and full connectivity (equivalently, no cycle). Checking only the edge count misses graphs that are disconnected-with-a-cycle-elsewhere; checking only "no cycle" misses a forest of several separate trees. The clean insight is that once you already know there are exactly `n - 1` edges, "no cycle" and "fully connected" become the *same* statement, so a single union-find pass (or DFS) that only watches for cycles is enough.

Example.

```
Input:  n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
Output: true

           0
         / | \
        1  2  3
        |
        4
```

Intuition (diagram). Contrast with a graph that has exactly `n - 1` edges but is **not** a tree — a cycle in one part forces a disconnected leftover somewhere else, because you've "spent" edges on the cycle instead of on reaching every node:

```
n = 5, edges = [[0,1],[1,2],[2,0],[3,4]]   (4 edges = n-1, but NOT a tree)

   0---1        3---4
    \ /
     2
```

Here nodes `{0,1,2}` form a cycle (wasting an edge that could have connected a 4th node), and `{3,4}` is a completely separate component — so even though the edge count matches `n - 1`, the graph is neither connected nor acyclic.

Approach (step by step)

Key idea / invariant: a graph on `n` nodes is a tree **iff** it has exactly `n - 1` edges **and** contains no cycle. Given exactly `n - 1` edges, "acyclic" and "fully connected" are equivalent — a cycle anywhere forces some other node to be unreachable, because there aren't enough remaining edges to both close the cycle and reach every node. So the algorithm reduces to: check the edge count once, then union-find every edge, failing fast the instant an edge connects two nodes that are already in the same component (a cycle).

1. If `edges.length != n - 1`, return `false` immediately — too few edges can't connect everything, too many guarantees a cycle.
2. Initialize `parent[i] = i` for every node (`n` singleton components).
3. For each edge `(u, v)`: find the root of `u`'s component and the root of `v`'s component.
4. If the roots are the **same**, `u` and `v` are already connected, so this edge closes a cycle — return `false`.
5. Otherwise, union the two components (attach one root under the other).
6. If every edge was processed without ever finding matching roots, and the edge count was already confirmed to be `n - 1`, the graph is connected and acyclic — return `true`.

Alternative (DFS): build an adjacency list, DFS from node `0` while remembering the parent you arrived from (so you don't treat the edge back to your own parent as a false cycle); if you ever reach an already-visited node that isn't your immediate parent, that's a real cycle. Afterward, confirm every node was visited (full connectivity). Union-find is iterative and avoids recursion-depth issues on a long skewed graph; DFS is often more intuitive to state.

Worked trace on `n = 5`, `edges = [[0,1],[0,2],[0,3],[1,4]]`:

edge	root(a)	root(b)	same root (cycle)?	action	components after
[0,1]	0	1	no	union	{0,1}, {2}, {3}, {4}
[0,2]	0	2	no	union	{0,1,2}, {3}, {4}
[0,3]	0	3	no	union	{0,1,2,3}, {4}
[1,4]	0	4	no	union	{0,1,2,3,4}

Edge count is `4 = n − 1 = 5 − 1`, and no edge ever found matching roots, so all nodes end up in one component with no cycle — return `true`.

Brute force

Build an adjacency list and DFS from node `0`, tracking the parent to avoid false cycles, then check every node was visited.

```
public boolean validTreeBrute(int n, int[][] edges) {
    if (edges.length != n - 1) return false;   // wrong edge count can't be a tree
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    for (int[] e : edges) {
        graph.get(e[0]).add(e[1]);
        graph.get(e[1]).add(e[0]);
    }
    boolean[] visited = new boolean[n];
    if (hasCycle(graph, 0, -1, visited)) return false;
    for (boolean v : visited) {
        if (!v) return false;   // some node was never reached -> disconnected
    }
    return true;
}

private boolean hasCycle(List<List<Integer>> graph, int node, int parent, boolean[] visitec
    visited[node] = true;
    for (int neighbor : graph.get(node)) {
        if (neighbor == parent) continue;        // skip the edge back to where we came frc
        if (visited[neighbor]) return true;       // reached an already-visited node -> cyc
        if (hasCycle(graph, neighbor, node, visited)) return true;
    }
    return false;
}
```

Optimized (union-find, cycle detection + implicit connectivity)

Union every edge; a repeated root means a cycle, and `n − 1` clean unions means one component.

```
public boolean validTree(int n, int[][] edges) {
    if (edges.length != n - 1) return false;   // too few/too many edges to be a tree
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;

    for (int[] edge : edges) {
        int rootA = find(parent, edge[0]);
        int rootB = find(parent, edge[1]);
        if (rootA == rootB) return false;      // already connected -> this edge closes a c
        parent[rootA] = rootB;                  // union the two components
    }
    return true;   // n-1 edges + no cycle found => fully connected tree
}

private int find(int[] parent, int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];   // path compression
        x = parent[x];
    }
    return x;
}
```

Version	Time	Space
DFS + connectivity check	O(V + E), recursive (stack depth up to V)	O(V + E)
Union-find	O(V + E) (near O(1) per op with path compression), iterative	O(V)

6. BFS & Backtracking (new)

The core idea. BFS explores a graph **layer by layer**, so the first time it reaches a target it has found a **shortest** path in an unweighted graph — that's why it's the tool for "fewest moves/steps/hops." Backtracking explores a **decision tree** of partial choices, building up a candidate incrementally and abandoning ("backtracking out of") any branch that can't lead to a valid answer, undoing state as it returns so siblings start clean.

Reach for it when: you see "fewest/minimum number of moves/steps/buses/levels" (BFS), or "generate/find all" combinations/permutations/paths that satisfy a constraint, especially over a grid or with explicit choose/undo structure (backtracking).

6.1 Minimum Knight Moves (LC 1197)

Problem. A knight starts on square `(0, 0)` of an **infinite** chessboard. Given a target square `(x, y)`, return the minimum number of knight moves needed to reach `(x, y)`. A knight moves in an L-shape: two squares in one direction and one square perpendicular (8 possible moves from any square).

- **Constraints:** `-300 <= x, y <= 300`; the board is conceptually infinite (no edges to fall off), but `x, y` are bounded for the problem's test cases.
- **Clarifying assumptions:** the start is always `(0, 0)`; negative coordinates are allowed, so the search must handle all four quadrants (or exploit symmetry to avoid them).
- **Why it's tricky:** the board is unbounded, so a naive BFS has no natural frontier to stop at — without a bound on how far to search, visited-set growth looks unconstrained. The fix is symmetry: the knight's move set is symmetric under sign flips, so the minimum move count to `(x, y)` equals the move count to `(|x|, |y|)`, letting you search only the first quadrant with a small, provable bound on how far coordinates can stray.

Example.

```
Input:  x = 2, y = 1
Output: 1          (one knight move: (0,0) -> (2,1))
```

Intuition (diagram). From any square, a knight has 8 possible destination offsets. BFS explores all squares reachable in 1 move, then all squares reachable in 2 moves, and so on — the first layer that contains the target gives the answer.

```
the 8 knight-move offsets from square K:

       -2,-1  -2,+1
  -1,-2   .      .   -1,+2
     .    .  K   .     .
  +1,-2   .      .   +1,+2
       +2,-1  +2,+1

BFS frontiers from (0,0) (M = squares reached in that many moves):
        y
        2  .  M2 .  M2 .
        1  M2 .  M1 .  M2
  ------ 0  .  M1  K  M1 .   x
       -1  M2 .  M1 .  M2
       -2  .  M2 .  M2 .
           -2 -1  0  1  2
```

Approach (step by step)

Key idea: treat every square as a graph node with an edge to each of its 8 knight-move neighbors, then run plain BFS from `(0, 0)` until the target is popped off the queue. Because BFS expands in order of distance, the first time the target is dequeued, the recorded distance is the minimum number of moves.

Symmetry trick: the 8 offsets `(±1, ±2)` and `(±2, ±1)` are symmetric under negating either coordinate, so the distance to `(x, y)` is identical to the distance to `(|x|, |y|)`. Working in the first quadrant only keeps the visited set small and avoids ever needing a huge negative range.

1. Let `tx = |x|`, `ty = |y|` (fold the target into the first quadrant).
2. Initialize a queue with `(0, 0)` at distance `0`, and a visited set containing `(0, 0)`.
3. While the queue is non-empty: a. Pop `(cx, cy, dist)`. If `(cx, cy) == (tx, ty)`, return `dist`. b. For each of the 8 knight offsets, compute the neighbor `(nx, ny)`. To keep the search bounded, only enqueue neighbors with `nx >= −2` and `ny >= −2` (a knight moving from near the target quadrant never needs to wander more than 2 squares past 0 to loop back in) — if not visited, mark visited and enqueue at `dist + 1`.
4. The target is always reachable, so the loop always returns.

Worked trace on `(x, y) = (2, 1)`: - Fold: `tx = 2`, `ty = 1`. - Dequeue `(0,0,0)` — not target. Enqueue its 8 neighbors at distance 1, including `(2,1)`. - Dequeue `(2,1,1)` (or whichever neighbor comes off first; `(2,1)` is among the distance-1 set) — this **is** the target `(2, 1)` — return `1`. Matches the expected output.

Brute force

BFS without the symmetry fold, searching all four quadrants directly (correct, just wider).

```
public int minKnightMovesBrute(int x, int y) {
    int[][] moves = {{1,2},{2,1},{2,-1},{1,-2},{-1,-2},{-2,-1},{-2,1},{-1,2}};
    Set<Long> visited = new HashSet<>();
    Deque<int[]> queue = new ArrayDeque<>();
    queue.offer(new int[]{0, 0, 0});
    visited.add(0L);
    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        if (cur[0] == x && cur[1] == y) return cur[2];
        for (int[] m : moves) {
            int nx = cur[0] + m[0], ny = cur[1] + m[1];
            long key = ((long) nx << 20) ^ (ny & 0xFFFFF);   // pack signed coords into one
            if (visited.add(key)) {
                queue.offer(new int[]{nx, ny, cur[2] + 1});
            }
        }
    }
    return -1;   // unreachable in practice
}
```

Optimized (fold to first quadrant, bounded BFS)

Use symmetry so only non-negative-ish coordinates near the target quadrant are ever visited.

```
public int minKnightMoves(int x, int y) {
    int tx = Math.abs(x), ty = Math.abs(y);   // symmetry: distance only depends on |x|, |y
    int[][] moves = {{1,2},{2,1},{2,-1},{1,-2},{-1,-2},{-2,-1},{-2,1},{-1,2}};
    Set<Long> visited = new HashSet<>();
    Deque<int[]> queue = new ArrayDeque<>();
    queue.offer(new int[]{0, 0, 0});
    visited.add(0L);
    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        int cx = cur[0], cy = cur[1], dist = cur[2];
        if (cx == tx && cy == ty) return dist;
        for (int[] m : moves) {
            int nx = cx + m[0], ny = cy + m[1];
            if (nx < -2 || ny < -2) continue;              // stay near the target quadrant
            long key = ((long) (nx + 400) << 20) | (ny + 400);
            if (visited.add(key)) {
                queue.offer(new int[]{nx, ny, dist + 1});
            }
        }
    }
    return -1;   // unreachable in practice
}
```

Version	Time	Space
Unbounded BFS (all quadrants)	O(max(|x|,|y|)^2)	O(max(|x|,|y|)^2)
Symmetry-folded bounded BFS	O(max(|x|,|y|)^2) but smaller constant	O(max(|x|,|y|)^2)

6.2 Bus Routes (LC 815, Hard)

Problem. Given `routes[i]` — the list of bus stops that the `i`-th bus visits, in a loop, in order — and a `source` stop and `target` stop, return the least number of **buses** you must take to travel from `source` to `target`. You start on foot at `source`; return `-1` if it's not possible.

- **Constraints:** `1 <= routes.length <= 500`; `1 <= routes[i].length <= 10^5`; total stops across all routes `<= 10^5`; `0 <= routes[i][j] < 10^6`; a route's stops are distinct; `0 <= source, target < 10^6`.
- **Clarifying assumptions:** you can switch buses at any stop shared by two routes (no time penalty beyond "one more bus"); if `source == target` the answer is `0` (no bus needed).
- **Why it's tricky:** the natural graph is "stops connected by shared buses," but BFS-ing over individual stops doesn't directly count buses — you'd need to track which bus you're currently riding to know when a "hop" costs you a transfer. The clean fix is to **flip the modeling**: make BFS nodes be **routes** (buses), not stops, so each BFS layer naturally corresponds to "one more bus ridden."

Example.

```
Input:  routes = [[1,2,7],[3,6,7]], source = 1, target = 6
Output: 2          (take bus 0 from stop 1 to stop 7, transfer to bus 1, ride to stop 6)
```

Intuition (diagram). Build a map from each stop to the set of routes (buses) that visit it. Two routes are "adjacent" in the bus graph if they share at least one stop. BFS over routes: start from every route that visits `source` (distance 1 bus), and expand to any route that shares a stop with a route already visited.

```
stop -> routes:            route graph (edge = shared stop):
  1 -> {R0}                     R0 --- R1   (share stop 7)
  2 -> {R0}
  3 -> {R1}
  6 -> {R1}      <- target
  7 -> {R0, R1}  <- shared stop, the transfer point

BFS over routes: R0 (bus #1) -> R1 (bus #2, reaches stop 6) => answer 2
```

Approach (step by step)

Key idea: BFS nodes are **routes**, not stops. Build `stopToRoutes: stop -> list of route indices` that visit it. Start the BFS frontier with every route that visits `source`, each at "buses taken = 1." Expand a route `r` to every *other, unvisited* route `r2` that shares any stop with `r`. The first route layer that contains a route visiting `target` gives the answer.

1. If `source == target`, return `0` immediately (no bus needed).
2. Build `stopToRoutes`: for each route index `i`, for each stop `s` in `routes[i]`, append `i` to `stopToRoutes[s]`.
3. Initialize a queue with every route index that visits `source`, each at distance `1`; mark those routes visited.
4. While the queue is non-empty: a. Pop route `r` at distance `d`. If `routes[r]` contains `target`, return `d`. b. For each stop `s` in `routes[r]`, for each route `r2` in `stopToRoutes[s]` that hasn't

been visited yet: mark visited, enqueue `r2` at distance `d + 1` (one more bus).
5. If the queue empties without finding `target`, return `-1`.

Worked trace on `routes = [[1,2,7],[3,6,7]], source = 1, target = 6`: - `stopToRoutes`: `1->{R0}, 2->{R0}, 3->{R1}, 6->{R1}, 7->{R0,R1}`. - Routes visiting `source=1`: `{R0}`. Enqueue `R0` at distance 1, mark visited. - Dequeue `R0` (dist 1): stops `{1,2,7}`; does `R0` contain target `6`? No. - Expand via stop 1 -> routes `{R0}` (already visited, skip). - Expand via stop 2 -> routes `{R0}` (skip). - Expand via stop 7 -> routes `{R0, R1}`; `R1` unvisited -> enqueue `R1` at distance 2. - Dequeue `R1` (dist 2): stops `{3,6,7}`; contains target `6` -> return `2`. Matches expected.

Brute force

BFS over individual **stops**, tracking which buses have been boarded so far at each stop to avoid re-expanding the same bus for free; this still works but is easiest to reason about by just re-scanning all routes at each stop (accepting the extra work).

```
public int numBusesToDestinationBrute(int[][] routes, int source, int target) {
    if (source == target) return 0;
    int n = routes.length;
    List<Set<Integer>> routeStops = new ArrayList<>();
    for (int[] r : routes) {
        Set<Integer> s = new HashSet<>();
        for (int stop : r) s.add(stop);
        routeStops.add(s);
    }
    Set<Integer> visitedRoutes = new HashSet<>();
    Set<Integer> visitedStops = new HashSet<>();
    Deque<Integer> stopQueue = new ArrayDeque<>();
    stopQueue.offer(source);
    visitedStops.add(source);
    int buses = 0;
    while (!stopQueue.isEmpty()) {
        buses++;
        int size = stopQueue.size();
        for (int k = 0; k < size; k++) {
            int stop = stopQueue.poll();
            for (int i = 0; i < n; i++) {
                if (visitedRoutes.contains(i) || !routeStops.get(i).contains(stop)) continu
                visitedRoutes.add(i);
                for (int nextStop : routes[i]) {
                    if (nextStop == target) return buses;
                    if (visitedStops.add(nextStop)) stopQueue.offer(nextStop);
                }
            }
        }
    }
    return -1;
}
```

Optimized (BFS over routes via stop -> routes map)

Model buses as graph nodes so each BFS layer is exactly "one more bus."

```
public int numBusesToDestination(int[][] routes, int source, int target) {
    if (source == target) return 0;
    int n = routes.length;
    Map<Integer, List<Integer>> stopToRoutes = new HashMap<>();
    for (int i = 0; i < n; i++) {
        for (int stop : routes[i]) {
            stopToRoutes.computeIfAbsent(stop, k -> new ArrayList<>()).add(i);
        }
    }
    boolean[] visitedRoute = new boolean[n];
    Deque<Integer> queue = new ArrayDeque<>();
    for (int r : stopToRoutes.getOrDefault(source, Collections.emptyList())) {
        if (!visitedRoute[r]) {
            visitedRoute[r] = true;
            queue.offer(r);
        }
    }
    int buses = 1;
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int k = 0; k < size; k++) {
            int r = queue.poll();
            for (int stop : routes[r]) {
                if (stop == target) return buses;              // this bus reaches the targ
                for (int r2 : stopToRoutes.get(stop)) {
                    if (!visitedRoute[r2]) {
                        visitedRoute[r2] = true;               // one more bus to reach r2
                        queue.offer(r2);
                    }
                }
            }
        }
        buses++;
    }
    return -1;
}
```

Version	Time	Space
BFS over stops, rescan routes	O(n^2 · maxRouteLen)	O(n + stops)
BFS over routes (stop->routes map)	O(sum(routes[i].length) · avg routes-per-stop)	O(n + stops)

6.3 Generate Parentheses (LC 22)

Problem. Given `n` pairs of parentheses, generate all combinations of well-formed (balanced) parenthesis strings.

- **Constraints:** `1 <= n <= 8` (typical LeetCode bound — small because the output count grows combinatorially, the `n`-th Catalan number).
- **Clarifying assumptions:** order of the returned combinations doesn't matter; every string in the output has length exactly `2n`; no duplicates should appear.
- **Why it's tricky:** naively generating all `2^(2n)` strings of `(` / `)` and filtering for validity wastes enormous work on strings that go invalid early. The fix is to **prune during construction**: track how

many opens and closes have been placed so far and only ever add a character that keeps the prefix extendable into a valid string.

Example.

```
Input:  n = 2
Output: ["(())", "()()"]
```

Intuition (diagram). At each step you may add `(` if you haven't used all `n` opens yet, or `)` only if it wouldn't outnumber the opens placed so far (`close < open`). This keeps every partial string a valid *prefix* of some balanced string — no wasted branches.

```
recursion tree for n = 2   (state shown as open,close used)

                        (0,0)
                       (        <- close skipped: close(0) == open(0), can't close yet
                     (1,0)
                  (             )
              (2,0)           (1,1)
                 )              (             )
              (2,1)           (2,1)         (1,2) <- pruned: close > open, invalid
                 )              )
              (2,2)           (2,2)
              "(())"         "()()"
```

Approach (step by step)

Key idea / invariant: build the string left to right with backtracking, tracking `open` (count of `(` placed) and `close` (count of `)` placed). At every step, two choices are considered and each is only taken if it keeps the prefix valid: - add `(` only if `open < n` (haven't used up all opens yet); - add `)` only if `close < open` (never let closes outnumber opens placed so far).

When `open == close == n`, the string has length `2n` and is guaranteed balanced — record it.

1. Start a recursive `backtrack(current, open, close)` with an empty `current`, `open = 0`, `close = 0`.
2. If `current.length() == 2n`, add a copy of `current` to the result and return (base case).
3. If `open < n`: append `(`, recurse with `open + 1`, then remove the last character (backtrack — undo the choice so the sibling branch starts clean).
4. If `close < open`: append `)`, recurse with `close + 1`, then remove the last character (backtrack).
5. The two `if`s are independent, not `if/else` — both branches may fire from the same state, which is exactly how the tree branches.

Worked trace for `n = 2` (matches the diagram above): from `""` (0,0), only `(` is legal (close 0 is not `<` open 0) -> `"("` (1,0). From there both are legal: `(` -> `"(("` (2,0), or `)` -> `"()"` (1,1). From `"(("` (2,0) only `)` is legal (open is maxed) -> `"(())"` (2,1) -> `"(())"` (2,2), a full solution. From `"()"` (1,1) both fire again: `(` -> `"()("` (2,1) -> `")"` forced -> `"()()"` (2,2), a full solution; and `)` would need `close < open` i.e. `1 < 1`, false, so it's pruned. Final results: `["(())", "()()"]`, matching the expected output.

Brute force

Generate all `2^(2n)` bitstrings of `( / )` and validate each with a stack-style counter.

```
public List<String> generateParenthesisBrute(int n) {
    List<String> result = new ArrayList<>();
    int len = 2 * n;
    for (int mask = 0; mask < (1 << len); mask++) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < len; i++) {
            sb.append(((mask >> i) & 1) == 0 ? '(' : ')');
        }
        if (isValid(sb.toString())) result.add(sb.toString());
    }
    return result;
}

private boolean isValid(String s) {
    int balance = 0;
    for (char c : s.toCharArray()) {
        balance += c == '(' ? 1 : -1;
        if (balance < 0) return false;
    }
    return balance == 0;
}
```

Optimized (backtracking with open/close pruning)

Only ever extend a prefix that can still become balanced.

```
public List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    backtrack(result, new StringBuilder(), 0, 0, n);
    return result;
}

private void backtrack(List<String> result, StringBuilder current, int open, int close, int
    if (current.length() == 2 * n) {
        result.add(current.toString());     // full-length and guaranteed balanced by invar
        return;
    }
    if (open < n) {
        current.append('(');
        backtrack(result, current, open + 1, close, n);
        current.deleteCharAt(current.length() - 1);   // undo: try the sibling branch
    }
    if (close < open) {
        current.append(')');
        backtrack(result, current, open, close + 1, n);
        current.deleteCharAt(current.length() - 1);   // undo: try the sibling branch
    }
}
```

Version	Time	Space
Generate + validate all bitstrings	O(2^(2n) · n)	O(n) recursion/validation
Backtracking with pruning	O(4^n / sqrt(n)) (n-th Catalan number)	O(n) recursion depth

6.4 Word Search (LC 79)

Problem. Given an `m x n` grid of characters `board` and a string `word`, return `true` if `word` exists in the grid. The word must be built from letters of sequentially **adjacent** cells (horizontally or vertically neighboring), and the **same cell may not be used more than once** within one word.

- **Constraints:** `1 <= m, n <= 6` (typical LeetCode bound for this problem, kept small because worst case is exponential); `1 <= word.length <= 15`; board and word consist of uppercase and lowercase English letters.
- **Clarifying assumptions:** adjacency is 4-directional (up/down/left/right), not diagonal; each board cell can be reused across *different* attempted paths, just not twice within the *same* path.
- **Why it's tricky:** it looks like a simple grid search, but the "no reuse within one path" rule means you must track visited cells *per attempt* and undo that tracking when backing out of a branch — forgetting to unmark a cell after a failed path silently breaks later, otherwise-valid paths that would have reused it.

Example.

```
board = [["A","B","C","E"],
         ["S","F","C","S"],
         ["A","D","E","E"]]
word = "ABCCED"
Output: true
```

Intuition (diagram). Try starting a DFS from every cell that matches `word[0]`. At each step, look at the 4 neighbors for the next required letter, temporarily mark the current cell visited (so the path can't loop back onto itself), and unmark it on the way back out (backtrack) so other starting attempts can still use that cell.

```
board:                path for "ABCCED":
A  B  C  E            A* B* C* E
S  F  C  S            S  F  C* S
A  D  E  E            A  D  E* E*

* marks the cells used by the successful path, in order:
(0,0)A -> (0,1)B -> (0,2)C -> (1,2)C -> (2,2)E -> (2,3)E ... wait, check adjacency:
(0,0)A -> (0,1)B -> (0,2)C -> (1,2)C -> (2,2)E -> (2,1)D? no: word is A B C C E D
(0,0)A -> (0,1)B -> (0,2)C -> (1,2)C -> (2,2)E -> (2,1)D  matches "ABCCED"
```

Approach (step by step)

Key idea: DFS from every cell matching `word[0]`. At each recursive step, check the current cell against the next required character; if it matches, **mark it visited in place** (e.g. overwrite with a sentinel, or use a separate boolean grid), recurse into all 4 neighbors trying to match the next character, and **unmark it afterward** regardless of success — that unmark is the backtracking step that lets the cell be reused by a different path later.

Invariant: on entry to `dfs(row, col, idx)`, every cell currently marked "visited" lies on the path built so far for `word[0..idx-1]`, and `board[row][col]` has not yet been checked against `word[idx]`.

1. For each cell `(r, c)` in the grid, call `dfs(r, c, 0)`; if any call returns `true`, the word exists — return `true`.
2. `dfs(r, c, idx)`: a. If `idx == word.length()`, every character matched — return `true` (base case). b. If `(r, c)` is out of bounds, or already visited on this path, or `board[r][c] != word[idx]`, return `false` (pruning — this branch can't work). c. Mark `(r, c)` visited. d. Recurse into all 4 neighbors with `idx + 1`; if any neighbor call returns `true`, this branch succeeds. e. Unmark `(r, c)` (backtrack) before returning, so other paths can reuse this cell. f. Return whether any neighbor succeeded.

Worked trace on the example, starting from `(0,0)`, word `"ABCCED"`: | idx | cell | char needed | match? | action | |-----|---------|-------------|--------|--------| | 0 | (0,0) | A | yes | mark, recurse to neighbors | | 1 | (0,1) | B | yes | mark, recurse | | 2 | (0,2) | C | yes | mark, recurse | | 3 | (1,2) | C | yes | mark, recurse | | 4 | (2,2) | E | yes | mark, recurse | | 5 | (2,1) | D | yes | mark, `idx` reaches 6 -> return `true` |

All cells unwind returning `true` back up the call stack -> overall result `true`, matching the expected output.

Brute force

Explore all 4 directions with a separate `boolean[][] visited` array cloned per attempt is already close to optimal for this problem; the "brute" variant here instead recomputes a fresh visited array on every top-level starting cell rather than reusing in-place marking.

```
public boolean existBrute(char[][] board, String word) {
    int rows = board.length, cols = board[0].length;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            boolean[][] visited = new boolean[rows][cols];   // fresh per starting cell
            if (dfsBrute(board, word, r, c, 0, visited)) return true;
        }
    }
    return false;
}

private boolean dfsBrute(char[][] board, String word, int r, int c, int idx, boolean[][] vi
    if (idx == word.length()) return true;
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
    if (visited[r][c] || board[r][c] != word.charAt(idx)) return false;
    visited[r][c] = true;
    boolean found = dfsBrute(board, word, r + 1, c, idx + 1, visited)
        || dfsBrute(board, word, r - 1, c, idx + 1, visited)
        || dfsBrute(board, word, r, c + 1, idx + 1, visited)
        || dfsBrute(board, word, r, c - 1, idx + 1, visited);
    visited[r][c] = false;
    return found;
}
```

Optimized (in-place marking, no extra visited array)

Temporarily overwrite the visited cell with a sentinel instead of allocating a boolean grid.

```
public boolean exist(char[][] board, String word) {
    int rows = board.length, cols = board[0].length;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (dfs(board, word, r, c, 0)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] board, String word, int r, int c, int idx) {
    if (idx == word.length()) return true;                          // matched every charact
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length
        || board[r][c] != word.charAt(idx)) return false;           // bounds/mismatch prur
    char original = board[r][c];
    board[r][c] = '#';                                              // mark visited in plac
    boolean found = dfs(board, word, r + 1, c, idx + 1)
        || dfs(board, word, r - 1, c, idx + 1)
        || dfs(board, word, r, c + 1, idx + 1)
        || dfs(board, word, r, c - 1, idx + 1);
    board[r][c] = original;                                         // unmark: backtrack
    return found;
}
```

Version	Time	Space
DFS + fresh boolean grid per start	O(m·n · 4^L)	O(m·n) per attempt
DFS + in-place sentinel marking	O(m·n · 4^L)	O(L) recursion depth

(`L` = `word.length()`.)

7. Heap, Intervals & DP (new)

The core idea. Three techniques that constantly combine in interviews: a **heap** for efficiently merging several sorted sources or maintaining a running "best k"; an **interval sweep** for reasoning about overlaps and gaps across schedules; and **dynamic programming layered on sorted intervals**, where sorting turns "which earlier choice is compatible?" into a fast lookup (often binary search) instead of a linear scan.

Reach for it when: you see "k closest/smallest", multiple employees' or resources' busy/free schedules, "non-overlapping jobs maximizing profit/count", or any prompt where sorting plus a running boundary (max end, last compatible index) makes an otherwise quadratic scan collapse to `O(log n)` per step.

7.1 Find K Closest Elements (LC 658)

Problem. Given a sorted array of integers `arr`, an integer `k`, and an integer `x`, return the `k` elements of `arr` that are closest to `x`, sorted in **ascending order**. An element `a` is closer to `x` than `b` if `|a - x| < |b - x|`, or `|a - x| == |b - x|` and `a < b` (ties favor the smaller value).

- **Constraints:** `1 <= k <= arr.length`; `1 <= arr.length <= 10^4`; `arr` is sorted in ascending order (may contain duplicates); `-10^4 <= arr[i], x <= 10^4`.
- **Clarifying assumptions:** the output must be sorted ascending (not sorted by distance); `k` never exceeds `arr.length`, so a valid window always exists.
- **Why it's tricky:** because `arr` is sorted, the answer is always a **contiguous window** of size `k` — but finding the right window by scanning is `O(n)`, and it's easy to miss that the tie-break rule (prefer the smaller value) is exactly what makes a simple binary search on the window's left boundary correct.

Example.

```
Input:  arr = [1,2,3,4,5], k = 4, x = 3
Output: [1,2,3,4]
```

Intuition (diagram). Since `arr` is sorted, the `k` closest elements must sit together — you're really searching for the best **starting index** of a size-`k` window. Compare the window's left edge against the element just past its right edge: whichever is farther from `x` tells you which way to shift.

```
index:   0    1    2    3    4
arr:     1    2    3    4    5
                                  x = 3
start=0 window: [1  2  3  4] 5       arr[0+k]=arr[4]=5, x-arr[0]=2, arr[4]-x=2 -> tie, ke
start=1 window:  1 [2  3  4  5]      (only other valid start; n-k=1)
```

Approach (step by step)

Key idea / invariant: binary search directly on the window's left boundary `lo` in the range `[0, n-k]`. At each candidate `mid`, compare `x - arr[mid]` (how far the window's left edge is from `x`) against `arr[mid+k] - x` (how far the element just outside the window's right edge is from `x`). If the left edge is **strictly farther**, the window should not start at `mid` — shift right. Otherwise, `mid` is at least as good as anything to its right, so keep searching the left half (inclusive of `mid`).

1. Set `lo = 0`, `hi = arr.length - k` (the last index a size-`k` window can start at).
2. While `lo < hi`: a. `mid = lo + (hi - lo) / 2`. b. If `x - arr[mid] > arr[mid + k] - x`: `arr[mid]` is farther from `x` than the candidate just past the window, so the window must start later: `lo = mid + 1`. c. Else: `hi = mid` (this start is at least as good as anything to its right).
3. When `lo == hi`, the answer window is `arr[lo .. lo + k - 1]`.

Worked trace on `arr = [1,2,3,4,5]`, `k = 4`, `x = 3` (`n - k = 1`): - `lo = 0, hi = 1` → `mid = 0`: `x - arr[0] = 3 - 1 = 2`; `arr[4] - x = 5 - 3 = 2`. `2 > 2` is false → `hi = mid = 0`. - `lo == hi == 0` → window is `arr[0..3] = [1,2,3,4]`. Matches the expected output.

Heap alternative: push all elements into a max-heap of size `k` keyed by distance to `x` (or a min-heap over the whole array), which also works but costs `O(n log k)` and still needs a final sort — strictly worse here since the sortedness of `arr` already gives us a contiguous-window structure that binary search exploits directly.

Brute force

Sort every index by the official (distance, value) comparator, take the first `k`, then re-sort those `k` values ascending.

```
public List<Integer> findClosestElementsBrute(int[] arr, int k, int x) {
    Integer[] idx = new Integer[arr.length];
    for (int i = 0; i < arr.length; i++) idx[i] = i;
    // sort by distance to x, tie-break on the smaller value
    Arrays.sort(idx, (a, b) -> {
        int diff = Math.abs(arr[a] - x) - Math.abs(arr[b] - x);
        return diff != 0 ? diff : arr[a] - arr[b];
    });
    List<Integer> result = new ArrayList<>();
    for (int i = 0; i < k; i++) result.add(arr[idx[i]]);
    Collections.sort(result);
    return result;
}
```

Optimized (binary search on the window's left boundary)

Binary search directly for the window start instead of sorting anything.

```
public List<Integer> findClosestElements(int[] arr, int k, int x) {
    int lo = 0, hi = arr.length - k;   // hi = last valid window-start index
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        // compare how far the window's left edge is vs. the element just past its right ec
        if (x - arr[mid] > arr[mid + k] - x) {
            lo = mid + 1;    // left edge is farther from x -> shift the window right
        } else {
            hi = mid;        // this start is at least as good -> keep searching left half
        }
    }
    List<Integer> result = new ArrayList<>();
    for (int i = lo; i < lo + k; i++) result.add(arr[i]);
    return result;
}
```

Version	Time	Space
Sort by distance	O(n log n)	O(n)
Binary search on window	O(log(n-k) + k)	O(k)

7.2 Employee Free Time (LC 759, Hard)

Problem. Given a list of employees, each with a sorted, non-overlapping list of busy `Interval`s, return the list of finite intervals representing **common free time** for **all** employees, sorted by start time. (Free time stretching before the first meeting or after the last is infinite, so it is excluded.)

- **Constraints:** typical bounds: `1 <= schedule.length <= 50` employees, `0 <= intervals per employee <= 50`, `0 <= start < end <= 10^8`; each employee's own intervals are sorted and non-overlapping (but intervals *across* employees can overlap freely).
- **Clarifying assumptions:** an interval is common free time only if **every** employee is free during it; only return bounded (finite) gaps, never the unbounded ends.
- **Why it's tricky:** it looks like it needs pairwise comparison across employees, but the key insight is that common free time is exactly the **gaps in the union of everyone's busy intervals** — so the problem reduces to a standard "merge intervals, then report the gaps" sweep, as long as you correctly combine intervals from multiple sources.

Example.

```
Input:  employee1 = [[1,2],[5,6]]
        employee2 = [[1,3]]
        employee3 = [[4,10]]
Output: [[3,4]]
```

Intuition (diagram). Flatten (or merge) every employee's busy intervals into one timeline. Anywhere that no one is busy is free for everyone — that's a gap between the merged busy blocks.

```
time:   0  1  2  3  4  5  6  7  8  9  10
emp1:      [=]                 [=]
emp2:      [====]
emp3:               [================]
busy:      [====]   [================]
free:               ^^^
                     gap = [3,4]  <- the only common free interval
```

Approach (step by step)

Key idea / invariant: merge all employees' intervals into one time-ordered stream and sweep through it while tracking `maxEnd`, the farthest busy-interval end seen so far. If the next interval's start is **greater than** `maxEnd`, everyone is free during `[maxEnd, start)` — that gap is common free time. Otherwise the intervals overlap or touch, so just extend `maxEnd`.

1. Collect every interval from every employee into one list.
2. Order them by start time — either by flattening and sorting the full list, or by a `k`-way merge with a min-heap keyed on start time (one "current pointer" per employee).
3. Initialize `maxEnd` to the first interval's end.
4. For each subsequent interval `cur`, in start-time order: a. If `cur.start > maxEnd`, record the gap `[maxEnd, cur.start]` as free time. b. Update `maxEnd = max(maxEnd, cur.end)`.
5. Return the recorded gaps.

Worked trace (flattened + sorted by start: `[1,2], [1,3], [4,10], [5,6]`):

step	interval (by start)	start > maxEnd?	gap added	maxEnd after
1	[1,2]	— (first)	—	2
2	[1,3]	1 > 2 ? no	—	3
3	[4,10]	4 > 3 ? yes	[3,4]	10
4	[5,6]	5 > 10 ? no	—	10

Result: `[[3,4]]` — matches the expected output.

Brute force

Flatten every interval into one list, sort by start (`O(N log N)`), then sweep for gaps.

```
static class Interval {
    int start, end;
    Interval(int start, int end) { this.start = start; this.end = end; }
}

public List<Interval> employeeFreeTimeBrute(List<List<Interval>> schedule) {
    List<Interval> all = new ArrayList<>();
    for (List<Interval> employee : schedule) all.addAll(employee);
    all.sort((a, b) -> a.start - b.start);        // flatten, then sort by start time

    List<Interval> result = new ArrayList<>();
    int maxEnd = all.get(0).end;
    for (int i = 1; i < all.size(); i++) {
        Interval cur = all.get(i);
        if (cur.start > maxEnd) {                 // gap between merged busy blocks = free
            result.add(new Interval(maxEnd, cur.start));
        }
        maxEnd = Math.max(maxEnd, cur.end);
    }
    return result;
}
```

Optimized (k-way merge with a min-heap)

Merge across employees with a heap sized to the number of employees, rather than sorting every interval together — better when there are few employees but many intervals each.

```
static class Interval {
    int start, end;
    Interval(int start, int end) { this.start = start; this.end = end; }
}

public List<Interval> employeeFreeTime(List<List<Interval>> schedule) {
    // heap entry: {employeeIdx, intervalIdx}, ordered by that interval's start time
    PriorityQueue<int[]> heap = new PriorityQueue<>(
        (a, b) -> schedule.get(a[0]).get(a[1]).start - schedule.get(b[0]).get(b[1]).start);

    for (int e = 0; e < schedule.size(); e++) {
        if (!schedule.get(e).isEmpty()) heap.offer(new int[]{e, 0});   // seed each employe
    }

    List<Interval> result = new ArrayList<>();
    int maxEnd = -1;
    while (!heap.isEmpty()) {
        int[] top = heap.poll();
        Interval cur = schedule.get(top[0]).get(top[1]);
        if (maxEnd != -1 && cur.start > maxEnd) {
            result.add(new Interval(maxEnd, cur.start));   // gap since the last busy inter
        }
        maxEnd = Math.max(maxEnd, cur.end);
        if (top[1] + 1 < schedule.get(top[0]).size()) {
            heap.offer(new int[]{top[0], top[1] + 1});      // push this employee's next ir
        }
    }
    return result;
}
```

Version	Time	Space
Flatten + sort	O(N log N)	O(N)
K-way heap merge	O(N log k)	O(k)

(N = total intervals across all employees, k = number of employees.)

7.3 Maximum Profit in Job Scheduling (LC 1235)

Problem. You are given `n` jobs, each with a `startTime[i]`, `endTime[i]`, and `profit[i]`. Find the maximum profit achievable by scheduling a subset of jobs such that no two chosen jobs overlap in time (a job may start exactly when another ends — that's allowed, not an overlap).

- **Constraints:** `1 <= n <= 5*10^4`; `1 <= startTime[i] < endTime[i] <= 10^9`; `1 <= profit[i] <= 10^4`.
- **Clarifying assumptions:** you don't have to schedule every job; touching endpoints (`end == start` of the next job) count as compatible, not overlapping.
- **Why it's tricky:** the naive "try every subset" is exponential, and even a straightforward DP still needs, for each job, the best profit achievable among all **earlier, compatible** jobs — finding that quickly (instead of rescanning) is the whole point, and it requires sorting by **end time** (not start time) so that "compatible and already decided" lines up with "comes before" in the DP fill order.

Example.

```
Input:  startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]
Output: 120        (job [1,3] profit 50 + job [3,6] profit 70 = 120)
```

Intuition (diagram). Sort jobs by end time. For each job, either skip it, or take its profit plus the best profit achievable from jobs that already ended by the time this one starts. Jobs [1,3] and [3,6] don't overlap (they touch at 3), so both can be taken.

```
time:     0  1  2  3  4  5  6
J1 (50):     [====]
J2 (10):        [====]
J3 (40):           [======]
J4 (70):           [=========]
chosen:       [====]   [=========]
              J1(50)      J4(70)      -> 50 + 70 = 120
```

Approach (step by step)

Key idea / recurrence: sort jobs by end time. Let `dp[i]` = max profit using only the first `i` jobs in this end-time order. For job `i` (1-indexed), either skip it (`dp[i-1]`) or take it: `profit[i] + dp[p]`, where `p` is the count of earlier jobs whose end time is `<=` this job's start time (the most recent compatible job). Because the ends are sorted, `p` is found by **binary search** instead of a linear scan.

- **Recurrence:** `dp[i] = max(dp[i-1], profit[i] + dp[p_i])`.
- **Base case:** `dp[0] = 0` (no jobs taken, no profit).

- **Fill order:** ascending `i = 1..n` — `dp[i]` only depends on `dp[i−1]` and `dp[p_i]` with `p_i <= i−1`, both already computed.
- Sort jobs by `endTime`, producing parallel arrays `starts[]`, `ends[]`, `profits[]`.
- `dp[0] = 0`.
- For `i = 1..n`: a. Binary search `ends[0 .. i−2]` for the count `p` of entries `<= starts[i−1]` (the number of earlier jobs compatible with this one). b. `dp[i] = max(dp[i−1], profits[i−1] + dp[p])`.
- Return `dp[n]`.

Worked trace (sorted by end: `(1,3,50), (2,4,10), (3,5,40), (3,6,70)`, so `starts=[1,2,3,3]`, `ends=[3,4,5,6]`, `profits=[50,10,40,70]`):

i	job (start,end,profit)	search target	p (compatible count)	dp[i-1]	profit+dp[p]	dp[i]
1	(1,3,50)	start=1	0	0	50	50
2	(2,4,10)	start=2	0	50	10	50
3	(3,5,40)	start=3	1	50	40+50=90	90
4	(3,6,70)	start=3	1	90	70+50=120	120

`dp[4] = 120` — matches the expected output (jobs 1 and 4).

Brute force

Same DP, but find the latest compatible job with a **linear scan** instead of binary search.

```
public int jobSchedulingBrute(int[] startTime, int[] endTime, int[] profit) {
    int n = startTime.length;
    Integer[] order = new Integer[n];
    for (int i = 0; i < n; i++) order[i] = i;
    Arrays.sort(order, (a, b) -> endTime[a] - endTime[b]);   // sort jobs by end time

    int[] starts = new int[n], ends = new int[n], profits = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = startTime[order[i]];
        ends[i] = endTime[order[i]];
        profits[i] = profit[order[i]];
    }

    int[] dp = new int[n + 1];
    for (int i = 1; i <= n; i++) {
        int p = 0;
        for (int j = i - 1; j >= 1; j--) {          // linear scan for the latest compatibl
            if (ends[j - 1] <= starts[i - 1]) { p = j; break; }
        }
        dp[i] = Math.max(dp[i - 1], profits[i - 1] + dp[p]);
    }
    return dp[n];
}
```

Optimized (DP + binary search on end times)

Same recurrence, but binary search replaces the linear scan for the latest compatible job.

```java
public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
    int n = startTime.length;
    Integer[] order = new Integer[n];
    for (int i = 0; i < n; i++) order[i] = i;
    Arrays.sort(order, (a, b) -> endTime[a] - endTime[b]);   // process jobs in end-time or

    int[] starts = new int[n], ends = new int[n], profits = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = startTime[order[i]];
        ends[i] = endTime[order[i]];
        profits[i] = profit[order[i]];
    }

    int[] dp = new int[n + 1];
    for (int i = 1; i <= n; i++) {
        int p = latestCompatible(ends, i - 1, starts[i - 1]);   // count of jobs (0..i-2) w
        dp[i] = Math.max(dp[i - 1], profits[i - 1] + dp[p]);
    }
    return dp[n];
}

// binary search: count of entries in ends[0..limit-1] that are <= target
private int latestCompatible(int[] ends, int limit, int target) {
    int lo = 0, hi = limit;              // search only among jobs strictly before the curr
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (ends[mid] <= target) {
            lo = mid + 1;                // ends[mid] qualifies, look further right
        } else {
            hi = mid;
        }
    }
    return lo;                           // lo = count of qualifying jobs = dp index to use
}
```

Version	Time	Space
DP + linear scan	O(n^2)	O(n)
DP + binary search	O(n log n)	O(n)
